Mapping human lymph node cell types to 10X Visium with Cell2location - tutorial review

This tutorial shows how to use **cell2location** method for spatially resolving fine-grained cell types by integrating 10X Visium data with scRNA-seq reference of cell types. Cell2location is a principled Bayesian model that estimates which combination of cell types in which cell abundance could have given the mRNA counts in the spatial data, while modelling technical effects (platform/technology effect, contaminating RNA, unexplained variance).

Note!
Cell2location is an independent package, but is powered by scvi-tools. If you have questions about cell2location, Visium data or scvi-tools please visit https://discourse.scverse.org/c/ecosytem/cell2location/42, https://discourse.scverse.org/c/general/visium/32 or https://discourse.scverse.org/c/help/scvi-tools/7 correspondingly.

Cell2location

Open in Colab

In this tutorial, we analyse a publicly available Visium dataset of the human lymph node from 10X Genomics, and spatially map a comprehensive atlas of 34 reference cell types derived by integration of scRNA-seq datasets from human secondary lymphoid organs.

- Cell2location provides high sensitivity and resolution by borrowing statistical strength across locations. This is achieved by modelling similarity of location patterns between cell types using a hierarchical factorisation of cell abundance into tissue zones as a prior (see paper methods).

- Using our statistical method based on Negative Binomial regression to robustly combine scRNA-seq reference data across technologies and batches results in improved spatial mapping accuracy. Given cell type annotation for each cell, the corresponding reference cell type signatures $g_{f,g}$, which represent the average mRNA count of each gene g in each cell type f, can be estimated from sc/snRNA-seq data using either 1) NB regression or 2) a hard-coded computation of per-cluster average mRNA counts for individual genes. We generally recommend using NB regression. This notebook shows use a dataset composed on multiple batches and technologies.When the batch effects are small, a faster hard-coded method of computing per cluster averages provides similarly high accuracy. We also recommend the hard-coded method for non-UMI technologies such as Smart-Seq 2.

- Cell2location needs untransformed unnormalised spatial mRNA counts as input.

- You also need to provide cell2location with the expected average cell abundance per location which is used as a prior to guide estimation of absolute cell abundance. This value depends on the tissue and can be estimated by counting nuclei for a few locations in the paired histology image but can be approximate (see paper methods for more guidance).

Workflow diagram

see:
https://cell2location.readthedocs.io/en/latest/notebooks/cell2location_tutorial.html

Contents

- Loading packages
- Loading Visium and single cell data

1. Estimating cell type signatures (NB regression)
2. Cell2location: spatial mapping
3. Visualising cell abundance in spatial coordinates
4. Downstream analysis

- Leiden clustering of cell abundance
- Identifying cellular compartments / tissue zones using non-negative matrix factorisation (NMF)
- Estimate cell-type specific expression of every gene in the spatial data (needed for NCEM)

5. Advanced use

- Working with the posterior distribution and computing arbitrary quantiles

In [ ]:

Slide-seq

Slide-seq is a high-resolution spatial transcriptomics technology developed by researchers at the Broad Institute of MIT and Harvard (Fei Chen and Evan Macosko). It measures genome-wide gene expression within tissue sections while retaining the precise spatial location of cells. 1, 2, 3, 4

How It Works

- DNA-Barcoded Beads ("Pucks"): A dense monolayer of 10-micron DNA-barcoded beads (comparable to the size of individual cells) is packed onto a surface called a puck. 1, 2, 3
- RNA Transfer: Fresh-frozen tissue sections are placed directly onto the puck, allowing cellular RNA to transfer onto the underlying beads. 1, 2
- Sequencing & Mapping: Standard high-throughput RNA sequencing is performed. By matching the RNA sequences back to the known spatial coordinates of the bead

barcodes, researchers map gene expression across the tissue without needing a microscope for primary localization. 1, 2

Key Features & Versions

- Original Slide-seq (2019): Achieved 10-micron spatial resolution and enabled 3D tissue volume reconstruction (such as mapping the mouse hippocampus and cerebellum). 1, 2, 3
- Slide-seqV2 (2020): Improved bead synthesis, library generation, and array indexing to capture roughly 50% of single-cell RNA-seq efficiency—a ~10-fold increase in transcript capture over the original method. 1, 2
- Computational Pipeline: Processed using open-source, cloud-optimized workflows like the Broad Institute WARP Slide-seq Pipeline available on platforms like Terra. Commercialized assays are also known under tools like Curio Seeker. 1

In [ ]:

Loading packages

```
uv add scvi-colab
uv add cell2location
```

In [ ]:

In [1]:
```
import os, sys
import scanpy as sc
import numpy as np
import pandas as pd
from pathlib import Path

import scipy.sparse as sp
import anndata as ad
import scvi
import h5py

import cell2location
import squidpy as sq

'''
sed -i '1i import anndata' ~/uv/squidpy/.venv/lib/python3.12/site-package
head -5 ~/uv/squidpy/.venv/lib/python3.12/site-packages/cell2location/uti

import traceback
try:
    import cell2location
except Exception:
    traceback.print_exc()
'''

import matplotlib as mpl
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
from matplotlib import rcParams
from matplotlib.colors import LinearSegmentedColormap
```

```
rcParams['pdf.fonttype']= 42 # enables correct plotting of text for PDFs

import warnings

ROOT_SRC = Path("/home/flavio/uv/squidpy/src")
if str(ROOT_SRC) not in sys.path:
    sys.path.append(str(ROOT_SRC))

from libs.Basic import create_dir, pdwritecsv, pdreadcsv
from libs.matplotlib_figure_style import *
from libs.cell2location_lib import *          # clip_for_plot, create_gen

sc.logging.print_header()
print(f"squidpy=={sq.__version__}")
```

```
squidpy==0.1.dev34+gb2ed182e8.d20261020
```

First, let's define where we save the results of our analysis:

In [2]:
```
root0 = Path('/home/flavio/uv/squidpy')
root_results = root0 / 'results'
root_figure = root0 / 'figures'
root_data = root0 / 'data'

os.listdir(root_data)[:3]
```

Out[2]:
```
['visium_hne_liana_bivariate.h5ad',
 'sc_mouse_cortex.h5ad',
 'lymphoid_organ_sc.h5ad']
```

In [3]:
```
results_folder =  root_results / 'lymph_nodes_analysis/'

# create paths and names to results folders for reference regression and
ref_run_name = Path(f'{results_folder}/reference_signatures')
run_name = Path(f'{results_folder}/cell2location_map')
```

Loading Visium and scRNA-seq reference data

First let's read spatial Visium data from 10X Space Ranger output. Here we use lymph node data generated by 10X and presented in Kleshchevnikov et al (section 4, Fig 4). This dataset can be conveniently downloaded and imported using scanpy. See this tutorial for a more extensive and practical example of data loading (multiple visium samples).

In [4]:
```
import importlib

importlib.metadata.version('scanpy')
```

Out[4]:
```
'1.12.4'
```

In [5]:
```
# visium_sge will be deprecated
[x for x in sc.datasets.__dict__ if x.startswith('visium')]
```

Out[5]:
```
['visium_sge']
```

Visium data - adata_vis

In [6]:
```
# include_hires_tiff=True if you also want the full-resolution image

adata_vis = sc.datasets.visium_sge(sample_id="V1_Human_Lymph_Node")
adata_vis.obs['sample']= list(adata_vis.uns['spatial'].keys())[0]
```

```
/tmp/ipykernel_416266/4061658100.py:3: FutureWarning: The function visium_
sge is deprecated and will be removed in the future. Use :func:`squidpy.da
tasets.visium` instead.
  adata_vis = sc.datasets.visium_sge(sample_id="V1_Human_Lymph_Node")
/home/flavio/uv/squidpy/.venv/lib/python3.12/site-packages/scanpy/readwrit
e.py:248: UserWarning: Variable names are not unique. To make them unique,
call `.var_names_make_unique`.
  adata = adata.copy()
```

In [7]:
```
adata_vis
```

Out[7]:
```
AnnData object with n_obs × n_vars = 4035 × 36601
    obs: 'in_tissue', 'array_row', 'array_col', 'sample'
    var: 'gene_ids', 'feature_types', 'genome'
    uns: 'spatial'
    obsm: 'spatial'
    layers: None (.X)
```

In [8]:
```
print(adata_vis.var.shape[0])
adata_vis.var.head(3)
```

```
36601
```

Out[8]:

	gene_ids	feature_types	genome
MIR1302-2HG	ENSG00000243485	Gene Expression	GRCh38
FAM138A	ENSG00000237613	Gene Expression	GRCh38
OR4F5	ENSG00000186092	Gene Expression	GRCh38

In [9]:
```
adata_vis.uns['spatial'].keys()
```

Out[9]:
```
dict_keys(['V1_Human_Lymph_Node'])
```

In [10]:
```
adata_vis.obs.head(3)
```

Out[10]:

	in_tissue	array_row	array_col	sample
AAACAAGTATCTCCCA-1	1	50	102	V1_Human_Lymph_Node
AAACAATCTACTAGCA-1	1	3	43	V1_Human_Lymph_Node
AAACACCAATAACTGC-1	1	59	19	V1_Human_Lymph_Node

In [11]:
```
adata_vis.var_names[:5]
```

Out[11]:
```
Index(['MIR1302-2HG', 'FAM138A', 'OR4F5', 'AL627309.1', 'AL627309.3'], d
type='object')
```

In [12]:
```
adata_vis.var
```

Out[12]:

	gene_ids	feature_types	genome
MIR1302-2HG	ENSG00000243485	Gene Expression	GRCh38
FAM138A	ENSG00000237613	Gene Expression	GRCh38
OR4F5	ENSG00000186092	Gene Expression	GRCh38
AL627309.1	ENSG00000238009	Gene Expression	GRCh38
AL627309.3	ENSG00000239945	Gene Expression	GRCh38
...	...	...	...
AC141272.1	ENSG00000277836	Gene Expression	GRCh38
AC023491.2	ENSG00000278633	Gene Expression	GRCh38
AC007325.1	ENSG00000276017	Gene Expression	GRCh38
AC007325.4	ENSG00000278817	Gene Expression	GRCh38
AC007325.2	ENSG00000277196	Gene Expression	GRCh38

36601 rows × 3 columns

Note!
Here we rename genes to ENSEMBL ID for correct matching between single cell and spatial data - so you can ignore the scanpy suggestion to call `.var_names_make_unique`.

In [13]:
```
if 'symbol' not in adata_vis.var.columns:
    adata_vis.var.reset_index(inplace=True, names='symbol')

if 'gene_ids' in adata_vis.var.columns:
    adata_vis.var.set_index('gene_ids', inplace=True)

adata_vis.var.head(6)
```

Out[13]:

gene_ids	symbol	feature_types	genome
ENSG00000243485	MIR1302-2HG	Gene Expression	GRCh38
ENSG00000237613	FAM138A	Gene Expression	GRCh38
ENSG00000186092	OR4F5	Gene Expression	GRCh38
ENSG00000238009	AL627309.1	Gene Expression	GRCh38
ENSG00000239945	AL627309.3	Gene Expression	GRCh38
ENSG00000239906	AL627309.2	Gene Expression	GRCh38

In [ ]:
```
```

You can still plot gene expression by name using standard scanpy functions as follows:
**``python sc.pl.spatial(color='PTPRC', gene_symbols='symbol', ...)

```
<div class="alert alert-info">
<b>Note!</b>

Mitochondia-encoded genes (gene names start with prefix
mt- or MT-) are irrelevant for spatial mapping because
their expression represents technical artifacts in the
single cell and nucleus data rather than biological
abundance of mitochondria. Yet these genes compose 15-40%
of mRNA in each location. Hence, to avoid mapping
artifacts we strongly recommend removing mitochondrial
genes.
</div>
```

In [14]:
```
"CD109" in list(adata_vis.var.symbol)
```

Out[14]:
```
True
```

In [15]:
```
genes = [x for x in adata_vis.var.symbol if x.startswith("CD")]
genes.sort()
np.array(genes)[:20]
```

Out[15]:
```
array(['CD101', 'CD109', 'CD14', 'CD151', 'CD160', 'CD163', 'CD163L1',
       'CD164', 'CD164L2', 'CD177', 'CD180', 'CD19', 'CD1A', 'CD1B',
       'CD1C', 'CD1D', 'CD1E', 'CD2', 'CD200', 'CD200R1'], dtype='<U12')
```

B-cell geneS

Pan B-cell gene and protein markers used to identify B lymphocytes include CD19, MS4A1 (which encodes CD20), CD79A, CD79B, PAX5, and CD22. 1, 2, 3

Key B-Cell Gene and Surface Markers

- Core Pan B-Cell Markers:
  - CD19: Expressed from the early pro-B stage through mature B cells; acts as a coreceptor for B-cell receptor (BCR) signaling.
  - MS4A1 (CD20): Encodes the CD20 calcium channel protein; present from the pre-B stage to mature B cells and a primary target for immunotherapies (e.g., rituximab).
  - CD79A / CD79B: Encodes Igα and Igβ, essential signaling components associated with the surface B-cell receptor complex.
  - PAX5: A master transcription factor critical for B-cell commitment and identity.
  - CD22: A sialic acid-binding lectin expressed on mature and developing B cells. 1, 2, 3, 4, 5, 6, 7
- Stage-Specific and Subset Markers:
  - Naïve B Cells: IGHD (IgD), IGHM (IgM), CD27−, IL4R, FCER2 (CD23).
  - Memory B Cells: CD27+, CD24+, and switched or unswitched immunoglobulins (IgG, IgA, IgE).
  - Plasma Cells / Plasmablasts: Downregulate CD19 and MS4A1/CD20 while upregulating CD38, SDC1 (CD138), PRDM1 (Blimp-1), and XBP1. 1, 2, 3

In [ ]:

In [16]:
```
genes_plot = ["CD19", "MS4A1", "CD79A", "CD79B", "PAX5", "CD22", "CD52",
              "CD53", "CDS2", "IGHD", "IGHM", "CD27", "CD28",
              "LEF1", "CCR7", "SELL", "TCF7", "AICDA",]

genes_plot.sort()

print("Genes to plot", len(genes_plot))

genes = [x for x in genes_plot if x in list(adata_vis.var.symbol)]

len(genes), "; ".join(genes)
```

```
Genes to plot 18
```

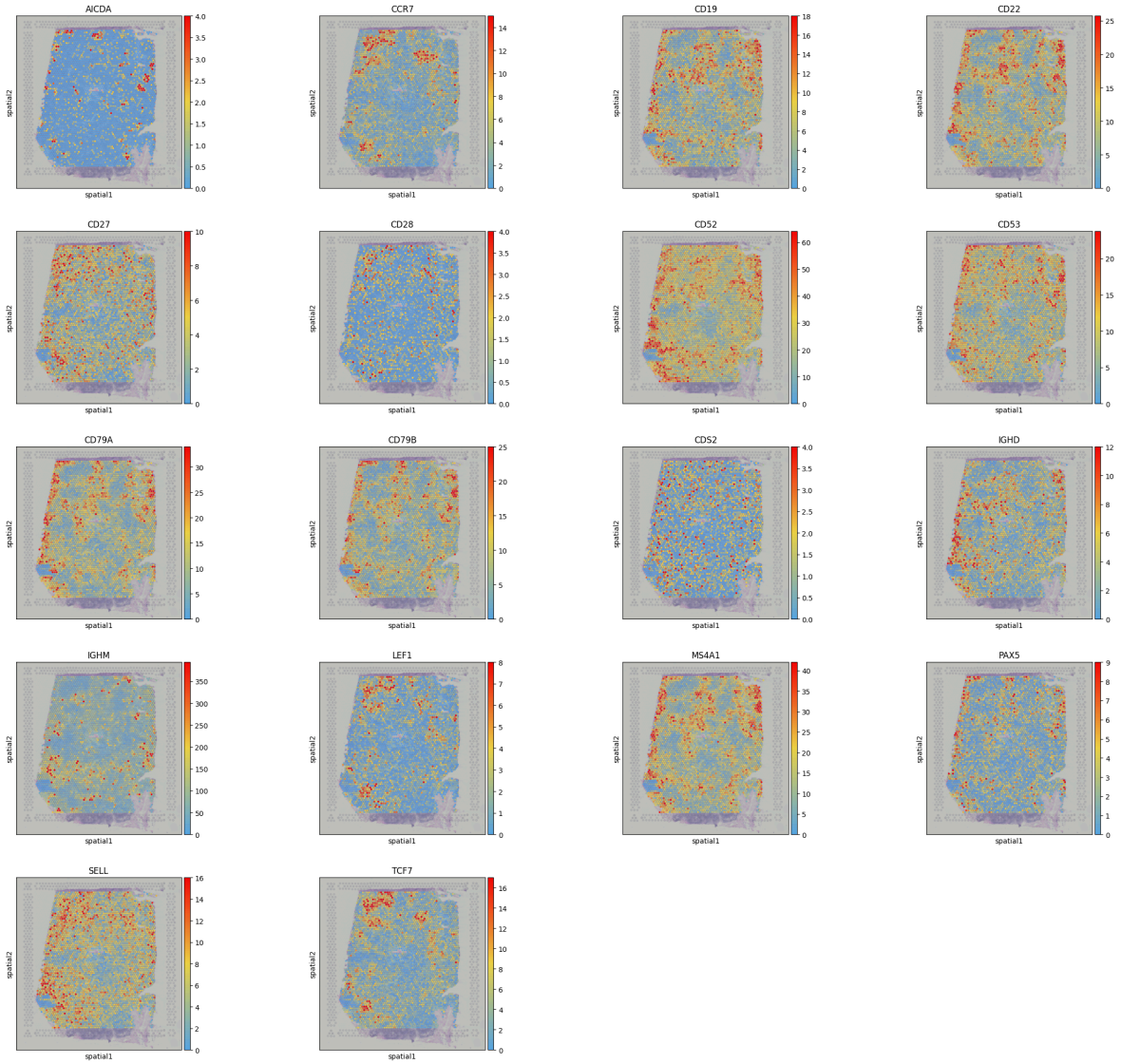
Out[16]:
```
(18,
 'AICDA; CCR7; CD19; CD22; CD27; CD28; CD52; CD53; CD79A; CD79B; CDS2; I
GHD; IGHM; LEF1; MS4A1; PAX5; SELL; TCF7')
```

Visium spatial plot per gene

- see filtered markers in: result/bcell_subset_unique_markers.csv

In [17]:
```
%matplotlib inline

# clip_for_plot / _values live in libs/cell2location_lib.py
# (warns about unknown features; non-zero percentile cap for very sparse
genes, cols = clip_for_plot(adata_vis, genes, q=99.2, vmin=0)
print(len(genes))

colors = ("#57a4e2", "#ebd246", "#f30707")
cmap_byr = LinearSegmentedColormap.from_list("blue_yellow_red", list(colo

sq.pl.spatial_scatter(
    adata_vis,
    color=cols,
    title=list(genes),            # show original names, not the _plot_ p
    img_res_key="hires",
    img_alpha=0.6,
    size=1.3,
    cmap=cmap_byr,
    ncols=4,
    library_id="V1_Human_Lymph_Node",
)
```

```
18
```



find mitochondria-encoded (MT) genes

In [18]:
```
adata_vis.var['MT_gene']= [gene.startswith('MT-') for gene in adata_vis.v

# remove MT genes for spatial mapping (keeping their counts in the object
adata_vis.obsm['MT']= adata_vis[:, adata_vis.var['MT_gene'].values].X.toa
adata_vis = adata_vis[:, ~adata_vis.var['MT_gene'].values]
```

Published scRNA-seq datasets of lymph nodes have typically lacked an adequate representation of germinal centre-associated immune cell populations due to age of patient donors. We, therefore, include scRNA-seq datasets spanning lymph nodes, spleen and tonsils in our single-cell reference to ensure that we captured the full diversity of immune cell states likely to exist in the spatial transcriptomic dataset.

Here we download this dataset, import into anndata and change variable names to ENSEMBL gene identifiers.

In [19]:
```
fname_sc = 'lymphoid_organ_sc.h5ad'
filename = root_data / fname_sc

# Read data
```

```
if filename.exists():
    adata_ref = ad.read_h5ad(filename)
    print(f"Data read from {filename}")
else:
    adata_ref = sc.read(
        filename,
        backup_url='https://cell2location.cog.sanger.ac.uk/paper/integrat
    )
    adata_ref.write_h5ad(filename)
    print(f"Data written to {filename}")


print(os.path.getsize(filename)/1e9,"GB")
print()

X=adata_ref.X
print(type(X), getattr(X,'dtype',None))

with h5py.File(filename,'r') as f:
    g=f['X']
    print(dict(g.attrs) if isinstance(g,h5py.Group) else g.shape)
    for k in f['layers'].keys() if 'layers' in f else []: print('layer',k
```

```
Data read from /home/flavio/uv/squidpy/data/lymphoid_organ_sc.h5ad
1.662152432 GB

<class 'scipy.sparse._csr.csr_matrix'> float32
{'encoding-type': 'csr_matrix', 'encoding-version': '0.1.0', 'shape': arra
y([73260, 10237])}
```

In [20]:
```
adata_ref
```

Out[20]:
```
AnnData object with n_obs × n_vars = 73260 × 10237
    obs: 'Age', 'BCELL_CLONE', 'BCELL_CLONE_SIZE', 'Donor', 'ID', 'IGH_MU_FREQ', 'ISOTYPE', 'LibraryID', 'Method', 'Population', 'PrelimCellType', 'Sample', 'Sex', 'Study', 'Tissue', 'barcode', 'batch', 'doublet_score', 'index', 'predicted_doublet', 'percent_mito', 'n_counts', 'n_genes', 'S_score', 'G2M_score', 'phase', 'VDJsum', 'cell_cycle_diff', 'PrelimCellType_new', 'leiden', 'leiden_1', 'leiden_2', 'leiden_3', 'leiden_4', 'CellType', 'CellType2', 'Subset', 'Subset_Broad', 'Subset_all', 'new_celltype', 'Subset_int', 'Subset_print'
    var: 'GeneID-2', 'GeneName-2', 'feature_types', 'feature_types-0', 'feature_types-1', 'gene_ids-1', 'gene_ids-4861STDY7135913-0', 'gene_ids-4861STDY7135914-0', 'gene_ids-4861STDY7208412-0', 'gene_ids-4861STDY7208413-0', 'gene_ids-Human_colon_16S7255677-0', 'gene_ids-Human_colon_16S7255678-0', 'gene_ids-Human_colon_16S8000484-0', 'gene_ids-Pan_T7935494-0', 'genome-1', 'n_cells', 'nonz_mean', 'mean_cov_effect_Subset_B_Cycling', 'mean_cov_effect_Subset_B_GC_DZ', 'mean_cov_effect_Subset_B_GC_LZ', 'mean_cov_effect_Subset_B_GC_prePB', 'mean_cov_effect_Subset_B_IFN', 'mean_cov_effect_Subset_B_activated', 'mean_cov_effect_Subset_B_mem', 'mean_cov_effect_Subset_B_naive', 'mean_cov_effect_Subset_B_plasma', 'mean_cov_effect_Subset_B_preGC', 'mean_cov_effect_Subset_DC_CCR7+', 'mean_cov_effect_Subset_DC_cDC1', 'mean_cov_effect_Subset_DC_cDC2', 'mean_cov_effect_Subset_DC_pDC', 'mean_cov_effect_Subset_Endo', 'mean_cov_effect_Subset_FDC', 'mean_cov_effect_Subset_ILC', 'mean_cov_effect_Subset_Macrophages_M1', 'mean_cov_effect_Subset_Macrophages_M2', 'mean_cov_effect_Subset_Mast', 'mean_cov_effect_Subset_Monocytes', 'mean_cov_effect_Subset_NK', 'mean_cov_effect_Subset_NKT', 'mean_cov_effect_Subset_T_CD4+', 'mean_cov_effect_Subset_T_CD4+_TfH', 'mean_cov_effect_Subset_T_CD4+_TfH_GC', 'mean_cov_effect_Subset_T_CD4+_naive', 'mean_cov_effect_Subset_T_CD8+_CD161+', 'mean_cov_effect_Subset_T_CD8+_cytotoxic', 'mean_cov_effect_Subset_T_CD8+_naive', 'mean_cov_effect_Subset_T_TIM3+', 'mean_cov_effect_Subset_T_TfR', 'mean_cov_effect_Subset_T_Treg', 'mean_cov_effect_Subset_VSMC', 'mean_sample_effectSample_4861STDY7135913', 'mean_sample_effectSample_4861STDY7135914', 'mean_sample_effectSample_4861STDY7208412', 'mean_sample_effectSample_4861STDY7208413', 'mean_sample_effectSample_4861STDY7462253', 'mean_sample_effectSample_4861STDY7462254', 'mean_sample_effectSample_4861STDY7462255', 'mean_sample_effectSample_4861STDY7462256', 'mean_sample_effectSample_4861STDY7528597', 'mean_sample_effectSample_4861STDY7528598', 'mean_sample_effectSample_4861STDY7528599', 'mean_sample_effectSample_4861STDY7528600', 'mean_sample_effectSample_BCP002_Total', 'mean_sample_effectSample_BCP003_Total', 'mean_sample_effectSample_BCP004_Total', 'mean_sample_effectSample_BCP005_Total', 'mean_sample_effectSample_BCP006_Total', 'mean_sample_effectSample_BCP008_Total', 'mean_sample_effectSample_BCP009_Total', 'mean_sample_effectSample_Human_colon_16S7255677', 'mean_sample_effectSample_Human_colon_16S7255678', 'mean_sample_effectSample_Human_colon_16S8000484', 'mean_sample_effectSample_Pan_T7935494'
    uns: 'Age_colors', 'Donor_colors', 'LibraryID_colors', 'Method_colors', 'Study_colors', 'Subset_Broad_colors', 'Subset_colors', 'Tissue_colors', 'leiden', 'neighbors', 'pca', 'phase_colors', 'regression_mod', 'umap'
    obsm: 'X_pca', 'X_umap'
    obsp: 'connectivities', 'distances'
    layers: None (.X)
```

In [21]:
```
print(adata_ref.var.shape)
list(adata_ref.var.columns)[:10]
```

```
(10237, 74)
```

Out[21]:
```
['GeneID-2',
 'GeneName-2',
 'feature_types',
 'feature_types-0',
 'feature_types-1',
 'gene_ids-1',
 'gene_ids-4861STDY7135913-0',
 'gene_ids-4861STDY7135914-0',
 'gene_ids-4861STDY7208412-0',
 'gene_ids-4861STDY7208413-0']
```

In [22]:
```
adata_ref.var.head(3)
```

Out[22]:

	GeneID-2	GeneName-2	feature_types	feature_types-0	feature_types-1
NOC2L	ENSG00000188976	16551	nan	Gene Expression	Gene Expression
HES4	ENSG00000188290	11070	nan	Gene Expression	Gene Expression
ISG15	ENSG00000187608	12375	nan	Gene Expression	Gene Expression

3 rows × 74 columns

In [23]:
```
if 'symbol' not in adata_ref.var.columns:
    adata_ref.var['symbol']= adata_ref.var.index
    # rename 'GeneID-2' as necessary for your data
    adata_ref.var.set_index('GeneID-2', drop=True, inplace=True)

    cols = list(adata_ref.var.columns)
    cols = ['symbol'] + cols[:-1]
    adata_ref.var = adata_ref.var[cols]

# delete unnecessary raw slot (to be removed in a future version of the t
del adata_ref.raw

adata_ref.var.head(3)
```

Out[23]:

GeneID-2	symbol	GeneName-2	feature_types	feature_types-0	feature_types-1
ENSG00000188976	NOC2L	16551	nan	Gene Expression	Gene Expression
ENSG00000188290	HES4	11070	nan	Gene Expression	Gene Expression
ENSG00000187608	ISG15	12375	nan	Gene Expression	Gene Expression

3 rows × 74 columns

lymphoid_organ_sc.h5ad - reference

`lymphoid_organ_sc.h5ad` is the single-cell reference for the cell2location paper's **lymph node / tonsil analysis**.

It holds **73,260 cells × 10,237 genes**, 1.66 GB on disk, and contains both raw counts and a fitted reference model. I opened it read-only in backed mode, so the file was not changed.

Expression matrix (`X`)

- **Raw UMI counts.** The matrix is sparse (CSR, float32) and every value is a whole number. In the first 2,000 cells the counts run from 1 to 1,276, with about 1,080 detected genes per cell.
- **Genes are already filtered.**
  - Gene symbols are the index and Ensembl IDs are in `var['GeneID-2']`.
  - The `n_counts` and `n_genes` columns were computed on the full gene set before filtering, so they are slightly higher than the row sums of `X` (e.g. 3,148 vs 3,122).
- **Cell QC:**
  - median 3,483 UMIs and 1,657 genes per cell;
  - minimum 581 UMIs and 698 genes.
  - Mitochondrial fraction (`percent_mito`) is 0–15%, median 4%.
  - Every cell has `predicted_doublet = False`, so doublets were already removed.

Studies and tissues (`obs`)

Study	Tissue	3′ (3GEX)	5′ (5GEX)
King_2020	Tonsil	2,002	27,703
James_2020	Mesenteric LN	10,250	4,184
Park_2020	Spleen / iLN / mLN / tLN	15,506	13,615

- **23 samples from 13 donors.** Donor A16 alone contributes 29,121 cells.
- **Ages** run from 0–4 years to 65–70 years.
- **Sex:** 48,545 female and 24,715 male cells.
- **Extra columns:** B-cell receptor fields (`BCELL_CLONE` , `ISOTYPE` , `IGH_MU_FREQ` , `VDJsum`), cell-cycle scores and phase, five Leiden clusterings, and several generations of annotation columns.

Cell-type annotations

- **`Subset` (34 types)** is the label set the regression model was trained on. The largest groups are B_mem (13,476), B_naive (8,924), T_CD4+_naive (6,012), B_Cycling (4,765) and T_CD4+_TfH (4,690). It also covers germinal-centre B cells (DZ, LZ, prePB, preGC), T-cell types (TfH, Tfr, Treg, CD8 types), NK/NKT/ILC, myeloid cells (DCs, macrophages, monocytes, mast cells) and stromal cells (Endo, FDC, VSMC). Rare types include ILC/Mast (18), VSMC (40) and DC_CCR7+ (42).

- `Subset_Broad` **(38 types)** is finer, despite its name. For example, it splits the endothelium into ACKR1+, CXCL12+ and lymphatic cells.
- **Likely label swap:** `Subset` has ILC = 617 and Mast = 18, while `Subset_Broad` has Mast = 617 and ILC = 18. I only compared the counts and did not cross-tabulate cell by cell. Check marker genes (e.g. TPSAB1/KIT for mast cells) before relying on either label for these two types.

Fitted reference model

This file already contains the trained cell2location reference model, so you don't need to retrain it.

- `uns['regression_mod']` is a `RegressionNBV4Torch` model with 57 factors: 34 cell types plus 23 sample effects. Its posterior means are:
  - `gene_factors` (57 × 10,237)
  - `sample_scaling` (23 × 1)
  - `tech_scaling` (2 × 10,237, one row per chemistry, 3′ and 5′)
  - `gene_E` (1 × 10,237)
- **In `var`:** 34 `mean_cov_effect_Subset_*` columns hold each cell type's expression signature. These are the columns to pass to cell2location as the reference. There are also 23 `mean_sample_effectSample_*` columns, plus `n_cells` and `nonz_mean`.

Embeddings and metadata

- **Embeddings:** `obsm['X_pca']` (100 components), `obsm['X_umap']`, and a neighbour graph built with 69 neighbours (`obsp` connectivities and distances).
- **Colour palettes** are stored in `uns` for the main categories.
- **Unusable columns:** `GeneName-2` holds numbers instead of gene names, and the `feature_types` / `gene_ids-*` columns are mostly NaN. Use the var index for gene names instead.

To extract the signatures for spatial mapping:

```
sig = adata.var[[c for c in adata.var if
c.startswith('mean_cov_effect_Subset_')]]
sig.columns = sig.columns.str.replace('mean_cov_effect_Subset_',
'')
```

This was a description only, so I didn't save any files.

In [24]:
```
obs=adata_ref.obs

for c in ['Study','Tissue','Method','Donor','Sample','Age','Sex','Subset_
    vc=obs[c].value_counts(dropna=False)
    print(f"\n## {c} ({vc.size} levels)")
    print(vc.head(40).to_string())

print(obs[['n_counts','n_genes','percent_mito','doublet_score']].describe

# X values sample
sub=adata_ref.X[:2000]
d=sub.data
```

```
print("\n")

print("X sample: min",d.min(),"max",d.max(),"integer?",np.allclose(d,np.r
print("rowsum vs n_counts", np.asarray(sub.sum(1)).ravel()[:5], obs.n_cou
print(adata_ref.var.index[:5].tolist())
print(adata_ref.uns['regression_mod'].keys() if hasattr(adata_ref.uns['re
print({k:(adata_ref.uns[k] if k!='leiden' else adata_ref.uns[k]) for k in

adata_ref.var[['GeneName-2','n_cells','nonz_mean']].head()
```

```
## Study (3 levels)
Study
King_2020     29705
Park_2020     29121
James_2020    14434

## Tissue (5 levels)
Tissue
Tonsil    29705
mLN       20480
Spleen     8174
iLN        7455
tLN        7446

## Method (2 levels)
Method
5GEX    45502
3GEX    27758

## Donor (13 levels)
Donor
A16     29121
BCP5    13323
290b     6067
BCP6     4531
BCP9     3867
BCP8     2914
390c     2876
298c     2358
BCP3     2083
BCP2     2002
302c     1825
417c     1308
BCP4      985

## Sample (23 levels)
Sample
BCP005_Total              13323
BCP006_Total               4531
4861STDY7462253            4286
4861STDY7462255            4283
4861STDY7528597            3888
BCP009_Total               3867
4861STDY7462256            3852
4861STDY7528600            3594
4861STDY7528599            3172
4861STDY7135914            3126
4861STDY7462254            3085
4861STDY7528598            2961
4861STDY7135913            2941
BCP008_Total               2914
Pan_T7935494               2876
BCP003_Total               2083
BCP002_Total               2002
Human_colon_16S8000484     1308
4861STDY7208413            1228
4861STDY7208412            1130
Human_colon_16S7255678     1125
BCP004_Total                985
Human_colon_16S7255677      700
```

```
## Age (7 levels)
Age
20-25y    29121
0-4y      16237
5-9y      13468
60-64y     6067
65-70y     4184
55-60y     2358
40-44y     1825

## Sex (2 levels)
Sex
female    48545
male      24715

## Subset_Broad (38 levels)
Subset_Broad
B_mem               11880
B_naive              9059
T_CD4+_naive         6012
T_CD4+_TfH           4690
B_Cycling            4603
B_activated          3979
T_CD8+_cytotoxic     3890
T_CD4+_TfH_GC        3653
T_CD4+               3059
T_Treg               2958
B_GC_DZ              2500
T_CD8+_CD161+        2294
T_CD8+_naive         2253
B_GC_IZ              2058
B_mem_FCRL4+         1660
B_GC_LZ              1240
B_plasma             1094
T_Tfr                1065
NK                    940
NKT                   896
Mast                  617
NK CX3CR1+            432
T_TIM3+               357
Endo_ACKR1+           314
Monocytes             306
DC_pDC                226
Endo_CXCL12+          180
DC_cDC2               173
T_Cycling             162
Endo_Lymphatic        128
Macrophages_M1        121
Macrophages_M2        110
DC_cDC1               101
FDC                    76
B_GC_prePB             74
DC_CCR7+               42
VSMC                   40
ILC                    18

## Subset (34 levels)
Subset
B_mem               13476
```

```
B_naive              8924
T_CD4+_naive         6012
B_Cycling            4765
T_CD4+_TfH           4690
T_CD8+_cytotoxic     3890
T_CD4+_TfH_GC        3653
B_activated          3575
B_GC_LZ              3298
T_CD4+               3059
T_Treg               2958
B_GC_DZ              2500
T_CD8+_CD161+        2294
T_CD8+_naive         2253
NK                   1372
B_plasma             1094
T_TfR                1065
NKT                   896
Endo                  622
ILC                   617
B_preGC               404
T_TIM3+               357
Monocytes             306
DC_pDC                226
B_IFN                 199
DC_cDC2               173
Macrophages_M1        121
Macrophages_M2        110
DC_cDC1               101
FDC                    76
B_GC_prePB             74
DC_CCR7+               42
VSMC                   40
Mast                   18

## phase (3 levels)
phase
G1     53232
S      16761
G2M     3267

## predicted_doublet (1 levels)
predicted_doublet
False    73260
       n_counts   n_genes  percent_mito  doublet_score
count  73260.00  73260.00      73260.00       73260.00
mean    4105.59   2029.96          0.05           0.09
std     2620.86    936.30          0.03           0.08
min      581.00    698.00          0.00           0.00
25%     2546.00   1330.00          0.03           0.03
50%     3483.00   1657.00          0.04           0.07
75%     4817.00   2499.00          0.06           0.13
max    19999.00   7932.00          0.15           0.59


X sample: min 1.0 max 1276.0 integer? True nnz/cell 1080.674
rowsum vs n_counts [3122. 3231. 3741. 2616. 2854.] [3148. 3266. 3782. 265
4. 2898.]
['ENSG00000188976', 'ENSG00000188290', 'ENSG00000187608', 'ENSG0000018689
1', 'ENSG00000186827']
dict_keys(['fact_names', 'mod_name', 'obs_names', 'post_sample_means', 'va
```

```
r_names'])
{'neighbors': {'params': {'method': 'umap', 'n_neighbors': 69}}}
```

Out[24]:

GeneID-2	GeneName-2	n_cells	nonz_mean
ENSG00000188976	16551	11327	1.119714
ENSG00000188290	11070	2553	1.784567
ENSG00000187608	12375	19155	1.651318
ENSG00000186891	31000	3268	1.719706
ENSG00000186827	30985	4869	2.005340

In [ ]:

Split by Age-group and Tissues

First I'll check how tissue and age overlap in the scRNA reference, and look in the squidpy project for the `genes` list and the `adata_vis` object.

Note: tissue and age are confounded. Tonsil is the only babies/young tissue, and spleen, iLN and tLN are all adult. Let me find the notebook that defines `genes` and `adata_vis`.

This Visium slide has no tissue/age labels, so I'll instead build the B-cell gene panel on the scRNA reference, coloring UMAPs by each gene split by tissue × age group with the same 99.2-percentile clipping. First, I'll load the figure-style skill.

Since CD20 isn't a valid gene symbol (the gene is MS4A1), I'll plot MS4A1 labelled as "MS4A1 (CD20)" instead. Next I'll build the grid with genes in rows and tissue × age groups in columns, using a shared colour scale per gene.

I found two issues to fix before saving: colliding column headers, and a blank CD14 column caused by its extreme sparsity flattening the 99.2-percentile clip to 0. I'll fix the headers and adjust sparse genes like CD14 to use the 99.2th percentile of non-zero values instead.

Yes, but only on the scRNA reference, not on the Visium slide. `V1_Human_Lymph_Node` is a single lymph node section from one donor, so its spots carry no tissue or age labels. Tissue and age exist only in `lymphoid_organ_sc.h5ad`. So I drew the same gene panel and the same 99.2-percentile clipping on the reference UMAP, with one column per tissue × age group.

B-cell gene panel on the scRNA reference UMAP, split by tissue and age group

Age groups. Your cut-offs map onto the reference's age bins like this:

Group	Age bins	Cells
babies	0–4y	16,237

Group	Age bins	Cells
young	5–9y	13,468
adult	20–25y	29,121
old adult	40–44y	1,825
elder	55–60y, 60–64y, 65–70y	12,609

Tissue and age are almost entirely confounded. Only 8 of the 25 tissue × age combinations contain cells:

- Tonsil exists only as babies and young.
- Spleen, iLN and tLN are adult only, and all come from the single donor A16.
- Only mLN spans three age groups: adult, old adult (one donor, 1,825 cells) and elder (four donors).

So a difference between columns is just as likely to come from tissue, study (King, Park or James) or donor as from age.

How the plot differs from your Visium version:

- **CD20 is replaced by MS4A1.** `CD20` is not a gene symbol, so your `clip_for_plot` drops it without a warning, and it is probably missing from your Visium plot too. I plotted `MS4A1`, labelled "MS4A1 (CD20)".
- **CD14 needed a different cap.** More than 99.2% of cells have zero *CD14*, so the 99.2-percentile cap was 0 and the row came out blank. For that gene I clipped at the 99.2th percentile of the non-zero values instead. The same problem can hit any sparse gene in your Visium helper.
- **Expression is normalised.** The reference holds raw counts from mixed 3′ and 5′ chemistry, so I coloured by log1p(counts per 10k), computed per cell. Each gene has one colour scale shared across all columns, so columns can be compared directly.
- **The colour map is sequential.** I used a light-grey-to-red ramp instead of `RdBu_r`, because expression has no meaningful midpoint for a diverging map to centre on.

If you want the split on the slide itself, one option is to build separate cell2location reference signatures per group and map each onto the Visium spots. Given the confounding, though, the result would mostly reflect tissue and study, not age.

Files are in the output tray:

- `bcell_genes_umap_by_tissue_age.png` — the figure above.
- `bcell_genes_by_tissue_age.csv` — per gene and group: cell count, % of cells expressing, and mean log1p(CP10k).

In [25]:
```
# mLN only: naive / memory / GC markers by age group (adult, old adult, e
# genes are looked up by symbol (var_names are Ensembl IDs after cell 34)
ml = adata_ref[adata_ref.obs["Tissue"] == "mLN"].copy()
ml.obs["age_group"] = ml.obs["Age"].map(age_group)          # age_group()

sc.pp.normalize_total(ml, target_sum=1e4)
sc.pp.log1p(ml)
```

```
mln_genes, mln_ids = resolve_gene_ids(ml, ["LEF1", "CCR7", "SELL", "TCF7"
Xm = ml[:, mln_ids].X
Xm = Xm.toarray() if sp.issparse(Xm) else np.asarray(Xm)

print(">> found genes", mln_genes, '\n')

df_mln = pd.DataFrame(Xm, columns=mln_genes, index=ml.obs_names)
df_mln["age_group"] = ml.obs["age_group"].values
mln_order = ["adult", "old adult", "elder"]

print(df_mln.groupby("age_group")[mln_genes].mean().reindex(mln_order).ro
print("\n")
print((df_mln[mln_genes] > 0).groupby(df_mln["age_group"]).mean().reindex

del ml, Xm
```

```
>> found genes ['LEF1', 'CCR7', 'SELL', 'TCF7', 'CD27', 'CD28', 'AICDA']

              LEF1   CCR7   SELL   TCF7   CD27   CD28  AICDA
age_group
adult        0.441  0.763  0.963  0.537  0.826  0.282  0.002
old adult    0.184  0.622  0.482  0.509  0.556  0.185  0.002
elder        0.413  0.813  0.816  0.629  0.634  0.253  0.001


              LEF1   CCR7   SELL   TCF7   CD27   CD28  AICDA
age_group
adult        0.259  0.426  0.543  0.334  0.518  0.190  0.001
old adult    0.113  0.376  0.317  0.336  0.369  0.115  0.002
elder        0.254  0.454  0.442  0.371  0.388  0.167  0.001
```

In [26]:

```
print(">>> df_mln is a subset from X")
print(df_mln.shape)
df_mln.head(3)
```

```
>>> df_mln is a subset from X
(20480, 8)
```

Out[26]:

	LEF1	CCR7	SELL	TCF7	CD27	CD28	A
AAACCTGGTAGAGCTG-1-4861STDY7135913	0.000000	0.000000	0.000000	2.002311	1.435816	1.435816	
AAACCTGGTGTCAATC-1-4861STDY7135913	1.409771	1.972696	1.972696	0.000000	0.000000	1.972696	
AAACCTGTCAGAGGTG-1-4861STDY7135913	0.000000	0.000000	0.000000	1.301031	0.000000	0.000000	

`CD20` is not a gene symbol in the reference (the gene is `MS4A1`), so the panel uses `MS4A1`, labelled "MS4A1 (CD20)".

Grid below: genes in rows, tissue × age groups in columns; colour = log1p(CP10k) clipped at the 99.2th percentile per gene, one scale per gene row (shared across columns).

Caveat: tissue and age are confounded (Tonsil = babies/young only; Spleen/iLN/tLN = adult, one donor; mLN = adult / old adult / elder), so column differences are not pure

age effects.

My selected genes - to observe them inside the lymphonode

In [27]:
```
found, ids = resolve_gene_ids(adata_ref, genes_plot)   # works with symbo
print(len(found), 'of', len(genes_plot), 'genes found')
";".join(found)
```

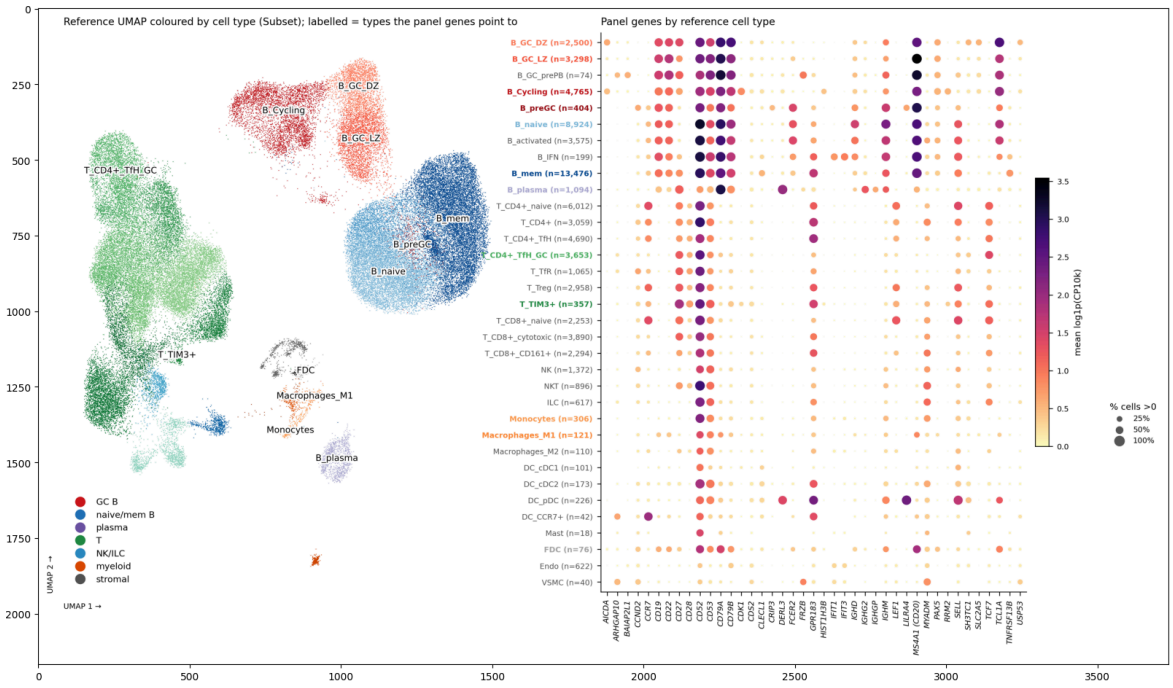
```
18 of 18 genes found
```

Out[27]:
```
'AICDA;CCR7;CD19;CD22;CD27;CD28;CD52;CD53;CD79A;CD79B;CDS2;IGHD;IGHM;LEF
1;MS4A1;PAX5;SELL;TCF7'
```

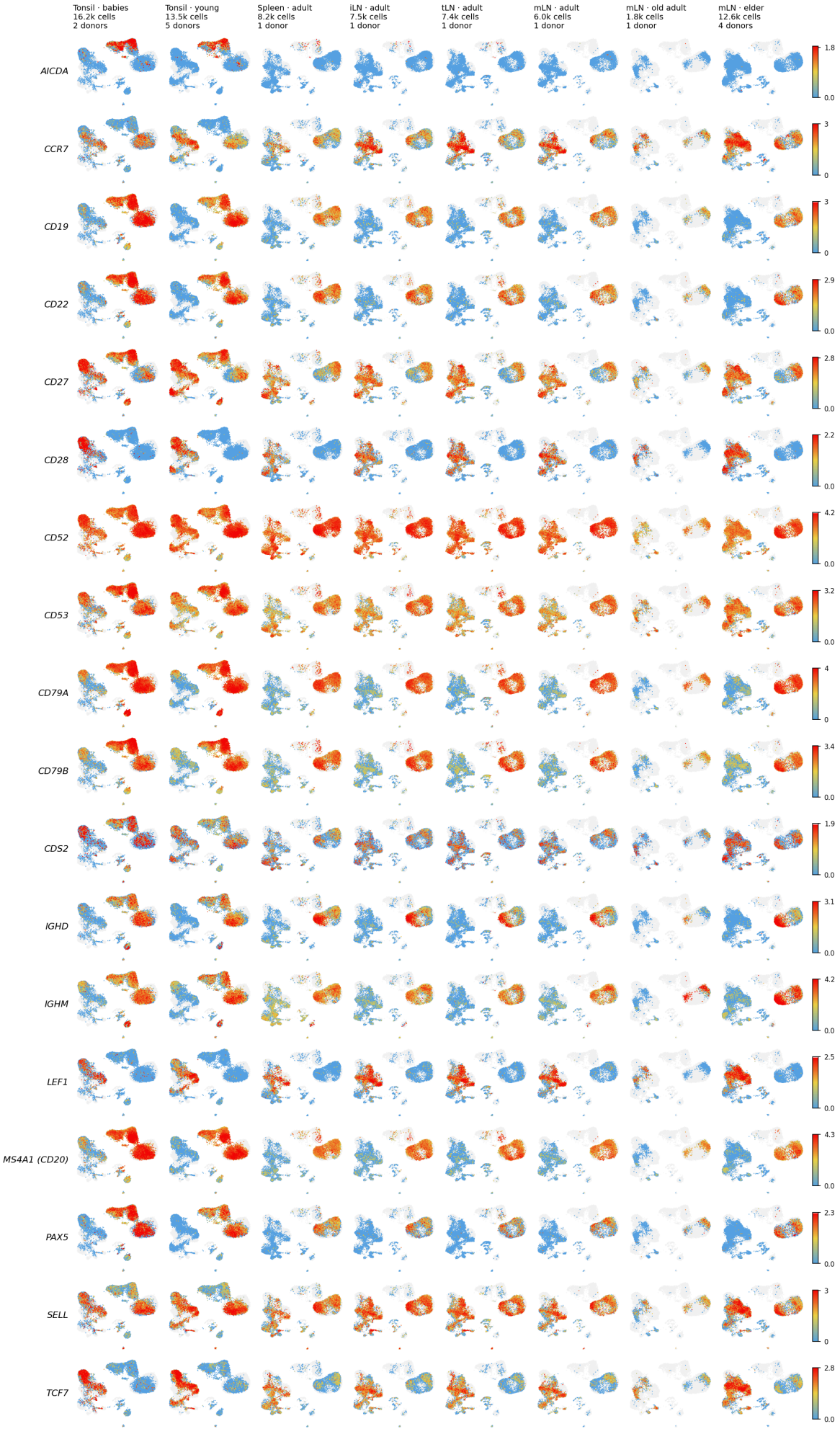
Below one calculate the UMAP's regions

- GC DZ: dark zone (DZ) of a germinal center (GC)
- GC LZ: germinal center (GC) light zone (LZ)
- preGC: Pre-germinal center (preGC) B cells
  - refer to B-cell subsets and precursor states that exist before they enter the germinal center (GC) reaction to undergo somatic hypermutation and class-switch recombination
- FDCs: Follicular dendritic cells
- B-cell cycling
  - refers to the process by which B lymphocytes transition from a resting state into active proliferation, DNA replication, and division during immune responses or development
- CD4+ Tfh (T follicular helper) cells are a specialized subset of CD4+ helper T cells
  - that reside primarily in lymphoid tissues and orchestrate germinal center (GC) reactions.
- TIM3+ refers to cells that express TIM-3 (T-cell immunoglobulin and mucin-domain containing-3)
  - an immune checkpoint protein found on various immune and tumor cells
  - Receptor Type: An inhibitory immune checkpoint receptor that acts as a negative regulator of the immune system.
  - Cell Expression: Found on exhausted CD8+ T cells, CD4+ T cells, regulatory T cells (Tregs), natural killer (NK) cells, M2 macrophages, and certain tumor cells.
  - Ligands: Interacts with molecules like galectin-9 (GAL-9), CEACAM1, HMGB1, and phosphatidylserine (PtdSer).

In [28]:
```
filefig = root_figure / "celltype_umap_dotplot.png"

if filefig.exists():
    plt.figure(figsize=(20,12))
    img = plt.imread(filefig)
    plt.imshow(img)
```



In [29]:
```
apply_figure_style(sizes=(8, 7, 6))

fig = plot_panel_umap_by_tissue_age(
    adata_ref, genes_plot,
    out_png = root_figure / 'bcell_genes_umap_by_tissue_age.png',
    labels={'MS4A1': 'MS4A1 (CD20)'},
    q=99.2, cmap_num=2
)
```


Tonsil · babies
16.2k cells
2 donors
Tonsil · young
13.5k cells
5 donors
Spleen · adult
8.2k cells
1 donor
iLN · adult
7.5k cells
1 donor
tLN · adult
7.4k cells
1 donor
mLN · adult
6.0k cells
1 donor
mLN · old adult
1.8k cells
1 donor
mLN · elder
12.6k cells
4 donors
AICDA
CCR7
CD19
CD22
CD27
CD28
CD52
CD53
CD79A
CD79B
CDS2
IGHD
IGHM
LEF1
MS4A1 (CD20)
PAX5
SELL
TCF7

Warning

Here we rename genes to ENSEMBL ID for correct matching between single cell and spatial data.

In [30]:
```
len(adata_ref.var.columns)
```

Out[30]: 74

In [31]:
```
adata_ref.var.head(6).T.head(12)
```

Out[31]:

GeneID-2	ENSG00000188976	ENSG00000188290	ENSG00000187608	E
symbol	NOC2L	HES4	ISG15	
GeneName-2	16551	11070	12375	
feature_types	nan	nan	nan	
feature_types-0	Gene Expression	Gene Expression	Gene Expression	
feature_types-1	Gene Expression	Gene Expression	Gene Expression	
gene_ids-1	ENSG00000188976	ENSG00000188290	ENSG00000187608	E
gene_ids-4861STDY7135913-0	ENSG00000188976	ENSG00000188290	ENSG00000187608	E
gene_ids-4861STDY7135914-0	ENSG00000188976	ENSG00000188290	ENSG00000187608	E
gene_ids-4861STDY7208412-0	ENSG00000188976	ENSG00000188290	ENSG00000187608	E
gene_ids-4861STDY7208413-0	ENSG00000188976	ENSG00000188290	ENSG00000187608	E
gene_ids-Human_colon_16S7255677-0	ENSG00000188976	ENSG00000188290	ENSG00000187608	E
gene_ids-Human_colon_16S7255678-0	ENSG00000188976	ENSG00000188290	ENSG00000187608	E

In [32]:
```
# var re-indexing (symbol column + Ensembl index) is done once in cell 34
# Repeating it here would overwrite var['symbol'] with Ensembl IDs and ra
assert "symbol" in adata_ref.var.columns and str(adata_ref.var_names[0]).
adata_ref.var[["symbol"]].head(3)
```

Out[32]:

GeneID-2	symbol
ENSG00000188976	NOC2L
ENSG00000188290	HES4
ENSG00000187608	ISG15

In [33]:
```
cols = ['symbol', 'GeneName-2', 'feature_types', 'feature_types-0', 'feat
       'gene_ids-1', 'gene_ids-4861STDY7135913-0',
       'genome-1', 'n_cells', 'nonz_mean', 'mean_cov_effect_Subset_B_Cycl
       'mean_cov_effect_Subset_B_GC_DZ',
       'mean_sample_effectSample_BCP009_Total',
       'mean_sample_effectSample_Human_colon_16S7255677',
       'mean_sample_effectSample_Human_colon_16S7255678',
       'mean_sample_effectSample_Human_colon_16S8000484',
       'mean_sample_effectSample_Pan_T7935494',]

adata_ref.var[cols].head(3).T
```

Out[33]:

GeneID-2	ENSG00000188976	ENSG0000018
symbol	NOC2L	H
GeneName-2	16551	1
feature_types	nan	
feature_types-0	Gene Expression	Gene Expres
feature_types-1	Gene Expression	Gene Expres
gene_ids-1	ENSG00000188976	ENSG0000018
gene_ids-4861STDY7135913-0	ENSG00000188976	ENSG0000018
genome-1	GRCh38	GR
n_cells	11327	2
nonz_mean	1.119714	1.784
mean_cov_effect_Subset_B_Cycling	0.370568	0.00
mean_cov_effect_Subset_B_GC_DZ	0.203758	0.000
mean_sample_effectSample_BCP009_Total	0.002198	0.00
mean_sample_effectSample_Human_colon_16S7255677	0.005277	0.238
mean_sample_effectSample_Human_colon_16S7255678	0.05186	0.416
mean_sample_effectSample_Human_colon_16S8000484	0.07157	0.0
mean_sample_effectSample_Pan_T7935494	0.00536	0.000

Note!
Before we estimate the reference cell type signature we recommend to perform very permissive genes selection. We prefer this to standard highly-variable-gene selection because our procedure keeps markers of rare genes while removing most of the uninformative genes.

The default parameters **cell_count_cutoff=5, cell_percentage_cutoff2=0.03, nonz_mean_cutoff=1.12** are a good starting point, however, you can increase the cut-off to exclude more genes.

To preserve marker genes of rare cell types we recommend low **cell_count_cutoff=5**, however, **cell_percentage_cutoff2** and **nonz_mean_cutoff** can be increased to select

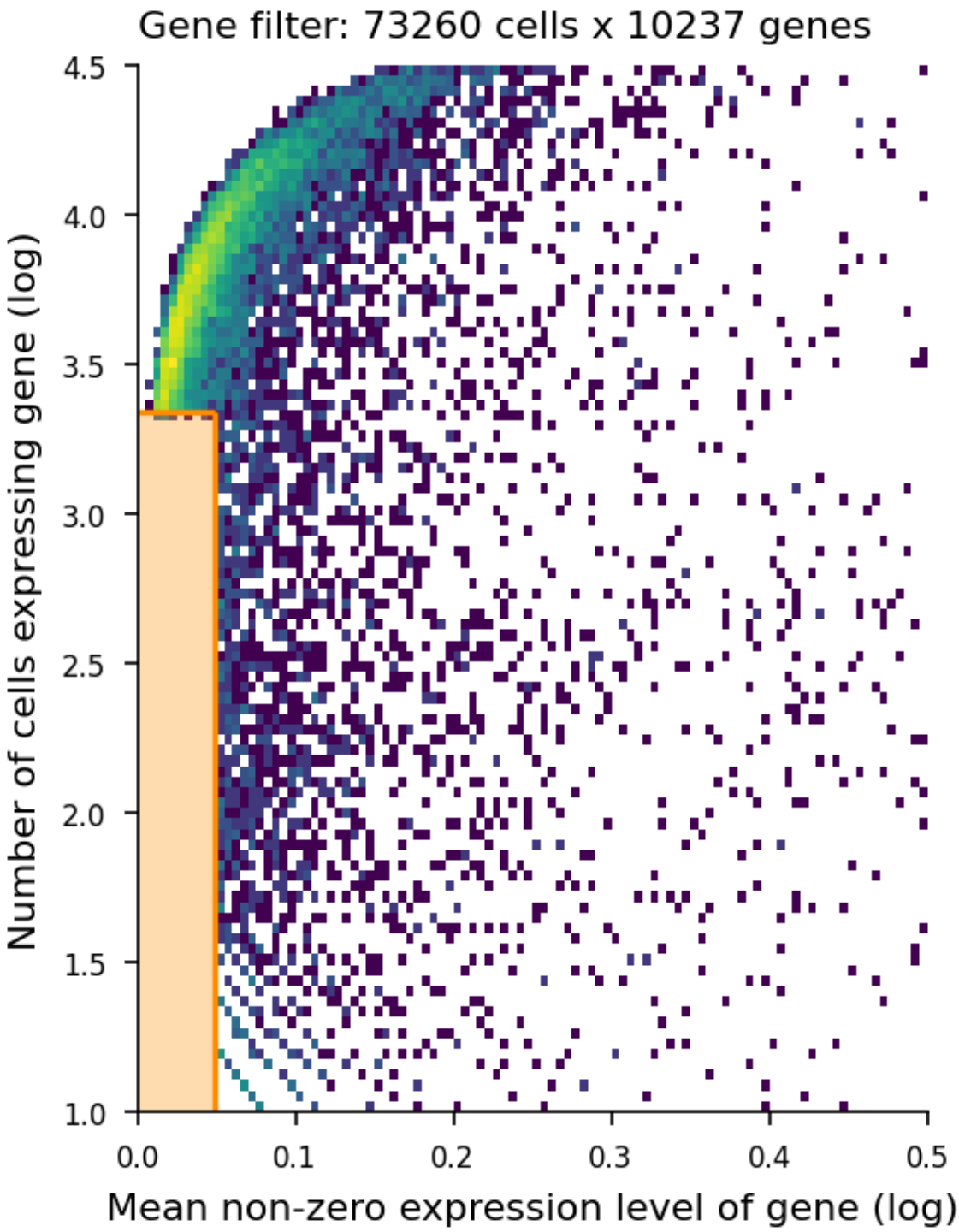
between 8k-16k genes.

In this 2D histogram, orange rectangle highlights genes excluded based on the combination of number of cells expressing that gene (Y-axis) and average RNA count for cells where the gene was detected (X-axis).

In this case, the downloaded dataset was already filtered using this method, hence no density under the orange rectangle (to be changed in the future version of the tutorial).

In [ ]:

In [34]:
```
from cell2location.utils.filtering import filter_genes

selected = filter_genes(adata_ref, cell_count_cutoff=5,
                        cell_percentage_cutoff2=0.03, nonz_mean_cutoff=1.
                        figsize=(3,4))

# filter the object
adata_ref = adata_ref[:, selected].copy()

len(selected), len(adata_ref), selected[:10]
```



Out[34]:
```
(10237,
 73260,
 Index(['ENSG00000188976', 'ENSG00000188290', 'ENSG00000187608',
        'ENSG00000186891', 'ENSG00000186827', 'ENSG00000078808',
        'ENSG00000160087', 'ENSG00000169972', 'ENSG00000224051',
        'ENSG00000162576'],
       dtype='object', name='GeneID-2'))
```

In [ ]:

Estimation of reference cell type signatures (NB regression)

The signatures are estimated from scRNA-seq data, accounting for batch effect, using a Negative binomial regression model.

Preparing anndata.

First, prepare anndata object for the regression model:

In [35]:
```
adata_ref.obs.head(3)
```

Out[35]:

	Age	BCELL_CLONE	BCELL_CLONE_SIZE	Donor	
AAACCTGGTAGAGCTG-1-4861STDY7135913	60-64y	nan	NaN	290b	4861STDY7135
AAACCTGGTGTCAATC-1-4861STDY7135913	60-64y	nan	NaN	290b	4861STDY7135
AAACCTGTCAGAGGTG-1-4861STDY7135913	60-64y	nan	NaN	290b	4861STDY7135

3 rows × 43 columns

In [36]:
```
adata_ref.obsm
```

Out[36]:
```
AxisArrays with keys: 'X_pca', 'X_umap'
```

In [37]:
```
adata_ref.var.head(3)
```

Out[37]:

	symbol	GeneName-2	feature_types	feature_types-0	feature_types-1
GeneID-2					
ENSG00000188976	NOC2L	16551	nan	Gene Expression	Gene Expression
ENSG00000188290	HES4	11070	nan	Gene Expression	Gene Expression
ENSG00000187608	ISG15	12375	nan	Gene Expression	Gene Expression

3 rows × 74 columns

In [38]:
```
adata_ref.var.shape
```

Out[38]:
```
(10237, 74)
```

In [39]:
```
adata_ref
```

Out[39]:
```
AnnData object with n_obs × n_vars = 73260 × 10237
    obs: 'Age', 'BCELL_CLONE', 'BCELL_CLONE_SIZE', 'Donor', 'ID', 'IGH_MU_FREQ', 'ISOTYPE', 'LibraryID', 'Method', 'Population', 'PrelimCellType', 'Sample', 'Sex', 'Study', 'Tissue', 'barcode', 'batch', 'doublet_score', 'index', 'predicted_doublet', 'percent_mito', 'n_counts', 'n_genes', 'S_score', 'G2M_score', 'phase', 'VDJsum', 'cell_cycle_diff', 'PrelimCellType_new', 'leiden', 'leiden_1', 'leiden_2', 'leiden_3', 'leiden_4', 'CellType', 'CellType2', 'Subset', 'Subset_Broad', 'Subset_all', 'new_celltype', 'Subset_int', 'Subset_print', 'age_group'
    var: 'symbol', 'GeneName-2', 'feature_types', 'feature_types-0', 'feature_types-1', 'gene_ids-1', 'gene_ids-4861STDY7135913-0', 'gene_ids-4861STDY7135914-0', 'gene_ids-4861STDY7208412-0', 'gene_ids-4861STDY7208413-0', 'gene_ids-Human_colon_16S7255677-0', 'gene_ids-Human_colon_16S7255678-0', 'gene_ids-Human_colon_16S8000484-0', 'gene_ids-Pan_T7935494-0', 'genome-1', 'n_cells', 'nonz_mean', 'mean_cov_effect_Subset_B_Cycling', 'mean_cov_effect_Subset_B_GC_DZ', 'mean_cov_effect_Subset_B_GC_LZ', 'mean_cov_effect_Subset_B_GC_prePB', 'mean_cov_effect_Subset_B_IFN', 'mean_cov_effect_Subset_B_activated', 'mean_cov_effect_Subset_B_mem', 'mean_cov_effect_Subset_B_naive', 'mean_cov_effect_Subset_B_plasma', 'mean_cov_effect_Subset_B_preGC', 'mean_cov_effect_Subset_DC_CCR7+', 'mean_cov_effect_Subset_DC_cDC1', 'mean_cov_effect_Subset_DC_cDC2', 'mean_cov_effect_Subset_DC_pDC', 'mean_cov_effect_Subset_Endo', 'mean_cov_effect_Subset_FDC', 'mean_cov_effect_Subset_ILC', 'mean_cov_effect_Subset_Macrophages_M1', 'mean_cov_effect_Subset_Macrophages_M2', 'mean_cov_effect_Subset_Mast', 'mean_cov_effect_Subset_Monocytes', 'mean_cov_effect_Subset_NK', 'mean_cov_effect_Subset_NKT', 'mean_cov_effect_Subset_T_CD4+', 'mean_cov_effect_Subset_T_CD4+_TfH', 'mean_cov_effect_Subset_T_CD4+_TfH_GC', 'mean_cov_effect_Subset_T_CD4+_naive', 'mean_cov_effect_Subset_T_CD8+_CD161+', 'mean_cov_effect_Subset_T_CD8+_cytotoxic', 'mean_cov_effect_Subset_T_CD8+_naive', 'mean_cov_effect_Subset_T_TIM3+', 'mean_cov_effect_Subset_T_TfR', 'mean_cov_effect_Subset_T_Treg', 'mean_cov_effect_Subset_VSMC', 'mean_sample_effectSample_4861STDY7135913', 'mean_sample_effectSample_4861STDY7135914', 'mean_sample_effectSample_4861STDY7208412', 'mean_sample_effectSample_4861STDY7208413', 'mean_sample_effectSample_4861STDY7462253', 'mean_sample_effectSample_4861STDY7462254', 'mean_sample_effectSample_4861STDY7462255', 'mean_sample_effectSample_4861STDY7462256', 'mean_sample_effectSample_4861STDY7528597', 'mean_sample_effectSample_4861STDY7528598', 'mean_sample_effectSample_4861STDY7528599', 'mean_sample_effectSample_4861STDY7528600', 'mean_sample_effectSample_BCP002_Total', 'mean_sample_effectSample_BCP003_Total', 'mean_sample_effectSample_BCP004_Total', 'mean_sample_effectSample_BCP005_Total', 'mean_sample_effectSample_BCP006_Total', 'mean_sample_effectSample_BCP008_Total', 'mean_sample_effectSample_BCP009_Total', 'mean_sample_effectSample_Human_colon_16S7255677', 'mean_sample_effectSample_Human_colon_16S7255678', 'mean_sample_effectSample_Human_colon_16S8000484', 'mean_sample_effectSample_Pan_T7935494'
    uns: 'Age_colors', 'Donor_colors', 'LibraryID_colors', 'Method_colors', 'Study_colors', 'Subset_Broad_colors', 'Subset_colors', 'Tissue_colors', 'leiden', 'neighbors', 'pca', 'phase_colors', 'regression_mod', 'umap'
    obsm: 'X_pca', 'X_umap'
    obsp: 'connectivities', 'distances'
    layers: None (.X)
```

prepare anndata for the regression model

In [40]:
```
cell2location.models.RegressionModel.setup_anndata(adata=adata_ref,
                        # 10X reaction / sample / batch
                        batch_key='Sample',
                        # cell type, covariate used for constructing sign
```

```
                        labels_key='Subset',
                        # multiplicative technical effects (platform, 3'
                        categorical_covariate_keys=['Method']
                       )
```

In [41]:
```
# create the regression model
from cell2location.models import RegressionModel
mod_reg = RegressionModel(adata_ref)
mod_reg
```

```
RegressionBackgroundDetectionTech model with the following params:
n_factors: 34
n_batch: 23
Training status: Not Trained
```

Out[41]:

Training model.

Now we train the model to estimate the reference cell type signatures.

Note that to achieve convergence on your data (=to get stabilization of the loss) you may need to increase `max_epochs=250` (See below).

Also note that here we are using `batch_size=2500` which is much larger than scvi-tools default and perform training on all cells in the data (`train_size=1`) - both parameters are defaults.

In [42]:
```
# view anndata_setup as a sanity check
mod_reg.view_anndata_setup()
```

```
Anndata setup with scvi-tools version 1.5.1.

Setup via `RegressionModel.setup_anndata` with arguments:
{
    'layer': None,
    'batch_key': 'Sample',
    'labels_key': 'Subset',
    'categorical_covariate_keys': ['Method'],
    'continuous_covariate_keys': None
}
```

Summary Statistics

Summary Stat Key	Value
n_batch	23
n_cells	73260
n_extra_categorical_covs	1
n_extra_continuous_covs	0
n_labels	34
n_vars	10237

Data Registry

Registry Key	scvi-tools Location
X	adata.X
batch	adata.obs['_scvi_batch']
extra_categorical_covs	adata.obsm['_scvi_extra_categorical_covs
ind_x	adata.obs['_indices']
labels	adata.obs['_scvi_labels']

batch State Registry

Source Location	Categories	scvi-tools Encoding
adata.obs['Sample']	4861STDY7135913	0
	4861STDY7135914	1
	4861STDY7208412	2
	4861STDY7208413	3
	4861STDY7462253	4
	4861STDY7462254	5
	4861STDY7462255	6
	4861STDY7462256	7
	4861STDY7528597	8
	4861STDY7528598	9
	4861STDY7528599	10
	4861STDY7528600	11
	BCP002_Total	12
	BCP003_Total	13
	BCP004_Total	14
	BCP005_Total	15
	BCP006_Total	16
	BCP008_Total	17
	BCP009_Total	18
	Human_colon_16S7255677	19
	Human_colon_16S7255678	20
	Human_colon_16S8000484	21
	Pan_T7935494	22

labels State Registry

Source Location	Categories	scvi-tools Encoding
adata.obs['Subset']	B_Cycling	0
	B_GC_DZ	1
	B_GC_LZ	2
	B_GC_prePB	3
	B_IFN	4
	B_activated	5
	B_mem	6
	B_naive	7
	B_plasma	8
	B_preGC	9
	DC_CCR7+	10
	DC_cDC1	11
	DC_cDC2	12
	DC_pDC	13
	Endo	14
	FDC	15
	ILC	16
	Macrophages_M1	17
	Macrophages_M2	18
	Mast	19
	Monocytes	20
	NK	21
	NKT	22
	T_CD4+	23
	T_CD4+_TfH	24
	T_CD4+_TfH_GC	25
	T_CD4+_naive	26
	T_CD8+_CD161+	27
	T_CD8+_cytotoxic	28
	T_CD8+_naive	29
	T_TIM3+	30
	T_TfR	31
	T_Treg	32
	VSMC	33

extra_categorical_covs State Registry

Source Location	Categories	scvi-tools Encoding
adata.obs['Method']	3GEX	0
	5GEX	1

In [43]:
```
adata_ref_filename = Path(f"{ref_run_name}/sc.h5ad")
adata_ref_filename.exists(), adata_ref_filename
```

Out[43]:
```
(True,
 PosixPath('/home/flavio/uv/squidpy/results/lymph_nodes_analysis/referen
ce_signatures/sc.h5ad'))
```

Training the model and exporting the posterior

In [44]:
```
if not adata_ref_filename.exists():
    mod_reg.train(max_epochs=250, accelerator='gpu')

    # In this section, we export the estimated cell abundance (summary of
```

```
    adata_ref = mod_reg.export_posterior(
        adata_ref, sample_kwargs={'num_samples': 1000, 'batch_size': 2500
    )

    # Save model
    mod_reg.save(f"{ref_run_name}", overwrite=True, save_anndata=True)
    print(f"Model saved with anndata to {ref_run_name}")

    # Save anndata object with results
    adata_ref.write(adata_ref_filename)
    print(f"Model saved to {adata_ref_filename}")
else:
    # Load AnnData with posterior results (already registered via setup_a
    # adata_ref = sc.read_h5ad(adata_ref_filename)

    # Reload the trained model bound to that AnnData
    mod_reg = cell2location.models.RegressionModel.load(f"{ref_run_name}"
    adata_ref = mod_reg.adata

    print(f"Model loaded from {ref_run_name}")
```

```
INFO     File /home/flavio/uv/squidpy/results/lymph_nodes_analysis/referen
ce_signatures/model.pt already downloaded
Model loaded from /home/flavio/uv/squidpy/results/lymph_nodes_analysis/ref
erence_signatures
```

In [ ]:

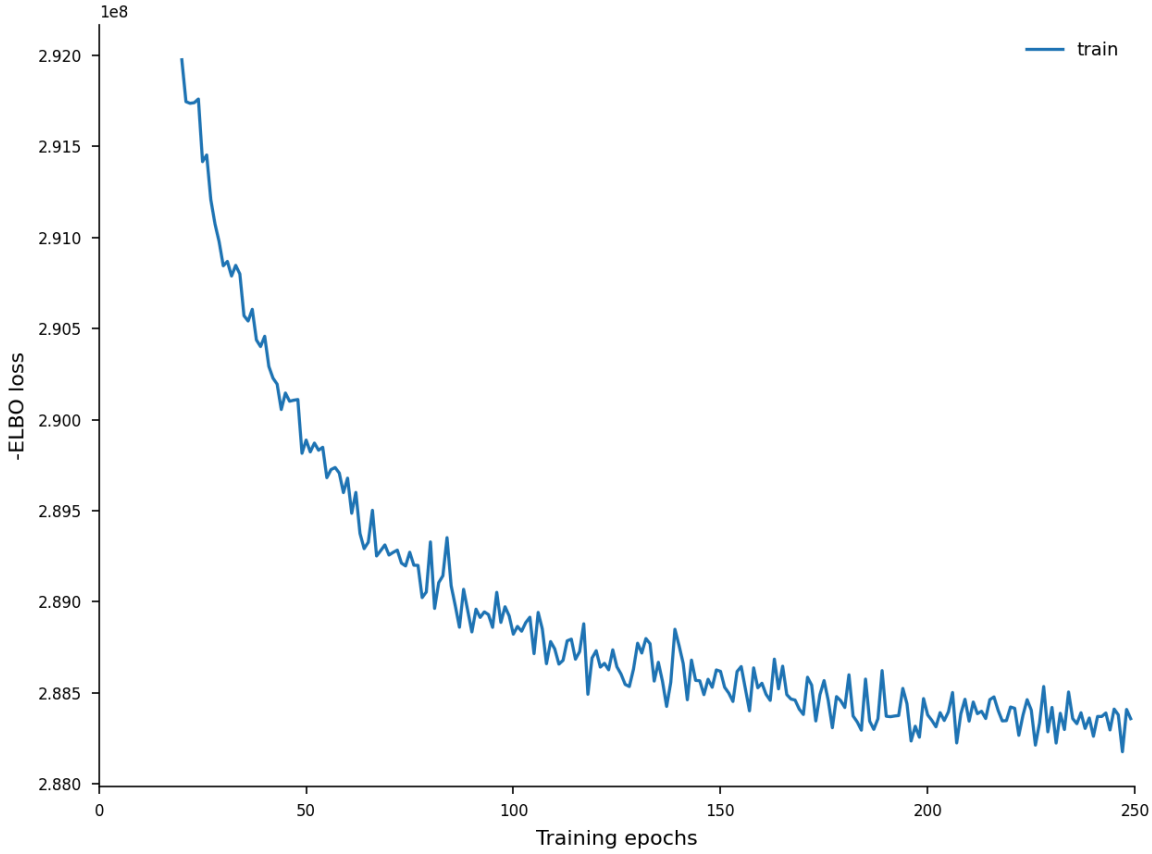
In [45]:
```
import torch

saved = torch.load(f"{ref_run_name}/model.pt", map_location="cpu", weight
model_genes = pd.Index(saved["var_names"])
adata_genes = adata_ref.var_names

print(len(model_genes), len(adata_genes))
print("missing from adata:", model_genes.difference(adata_genes)[:20].tol
print("extra in adata:", adata_genes.difference(model_genes)[:20].tolist(

del(saved)
```

```
10237 10237
missing from adata: []
extra in adata: []
```

Determine if the model needs more training.

Here, we plot ELBO loss history during training, removing first 20 epochs from the plot. This plot should have a decreasing trend and level off by the end of training. If it is still decreasing, increase `max_epochs`.

In [46]:
```
mod_reg.plot_history(20)
```



You can compute the 5%, 50% and 95% quantiles of the posterior distribution directly rather than using 1000 samples from the distribution (or any other quantiles). This speeds up application on large datasets and requires less memory - however, posterior mean and standard deviation cannot be computed this way.

In [47]:
```
list_q = ['q05_per_cluster_mu_fg', 'q50_per_cluster_mu_fg', 'q95_per_clus

ok = True if len([x for x in adata_ref.varm if x in list_q]) == len(list_

if not ok:
    adata_ref = mod_reg.export_posterior(
        adata_ref, use_quantiles=True,
        # choose quantiles
        add_to_varm=["q05","q50", "q95", "q0001"],
        sample_kwargs={'batch_size': 2500, 'accelerator': 'gpu'}
    )
else:
    print("export_posterior is ok: quantiles already exist in adata_ref.v
```

In [48]:
```
adata_ref.varm
```

Out[48]:
```
AxisArrays with keys: 'q05_per_cluster_mu_fg', 'q50_per_cluster_mu_fg',
'q95_per_cluster_mu_fg', 'q0001_per_cluster_mu_fg'
```

In [49]:
```
# picks 'means', else 'q50', else 'q05' (export_posterior(use_quantiles=T
inf_aver, summary = get_reference_signatures(adata_ref)
print(f"Using {summary} signatures")

inf_aver.head(6).T.head(4)
```

```
Using q50 signatures
```

Out[49]:

GeneID-2	ENSG00000188976	ENSG00000188290	ENSG00000187608	ENSG000001868
B_Cycling	0.433266	0.001882	0.384278	0.0198
B_GC_DZ	0.244424	0.000460	0.213292	0.0005
B_GC_LZ	0.311102	0.000511	0.274602	0.0558
B_GC_prePB	0.336622	0.037657	0.491036	0.0582

In [50]:
```
factor_names = adata_ref.uns['mod']['factor_names']
inf_aver = adata_ref.varm['q50_per_cluster_mu_fg'][
    [f'q50_per_cluster_mu_fg_{x}' for x in factor_names]
].copy()

inf_aver.columns = factor_names

inf_aver.head(6).T.head(4)
```

Out[50]:

GeneID-2	ENSG00000188976	ENSG00000188290	ENSG00000187608	ENSG000001868
B_Cycling	0.433266	0.001882	0.384278	0.0198
B_GC_DZ	0.244424	0.000460	0.213292	0.0005
B_GC_LZ	0.311102	0.000511	0.274602	0.0558
B_GC_prePB	0.336622	0.037657	0.491036	0.0582

Examine QC plots.

1. Reconstruction accuracy to assess if there are any issues with inference. This 2D histogram plot should have most observations along a noisy diagonal.

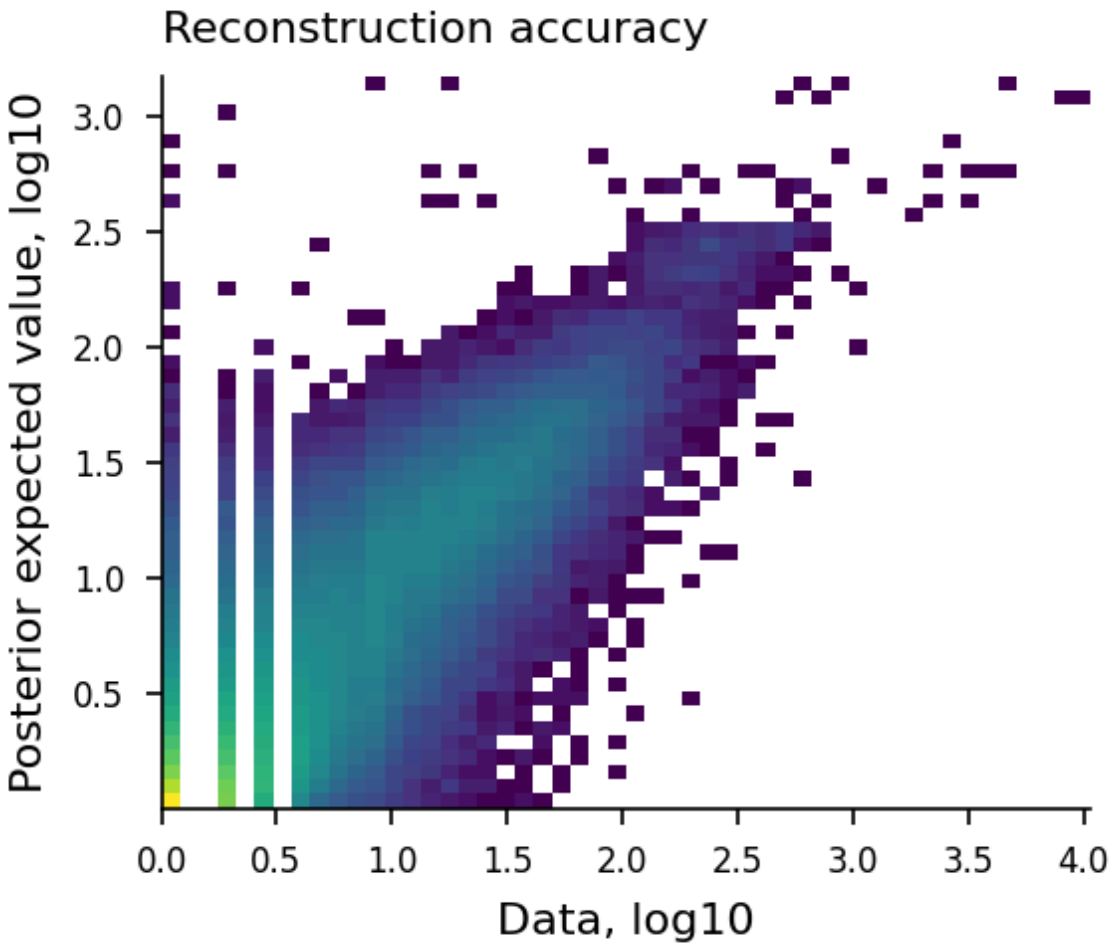
2. The estimated expression signatures are distinct from mean expression in each cluster because of batch effects. For scRNA-seq datasets which do not suffer from batch effect (this dataset does), cluster average expression can be used instead of estimating signatures with a model. When this plot is very different from a diagonal plot (e.g. very low values on Y-axis, density everywhere) it indicates problems with signature estimation.

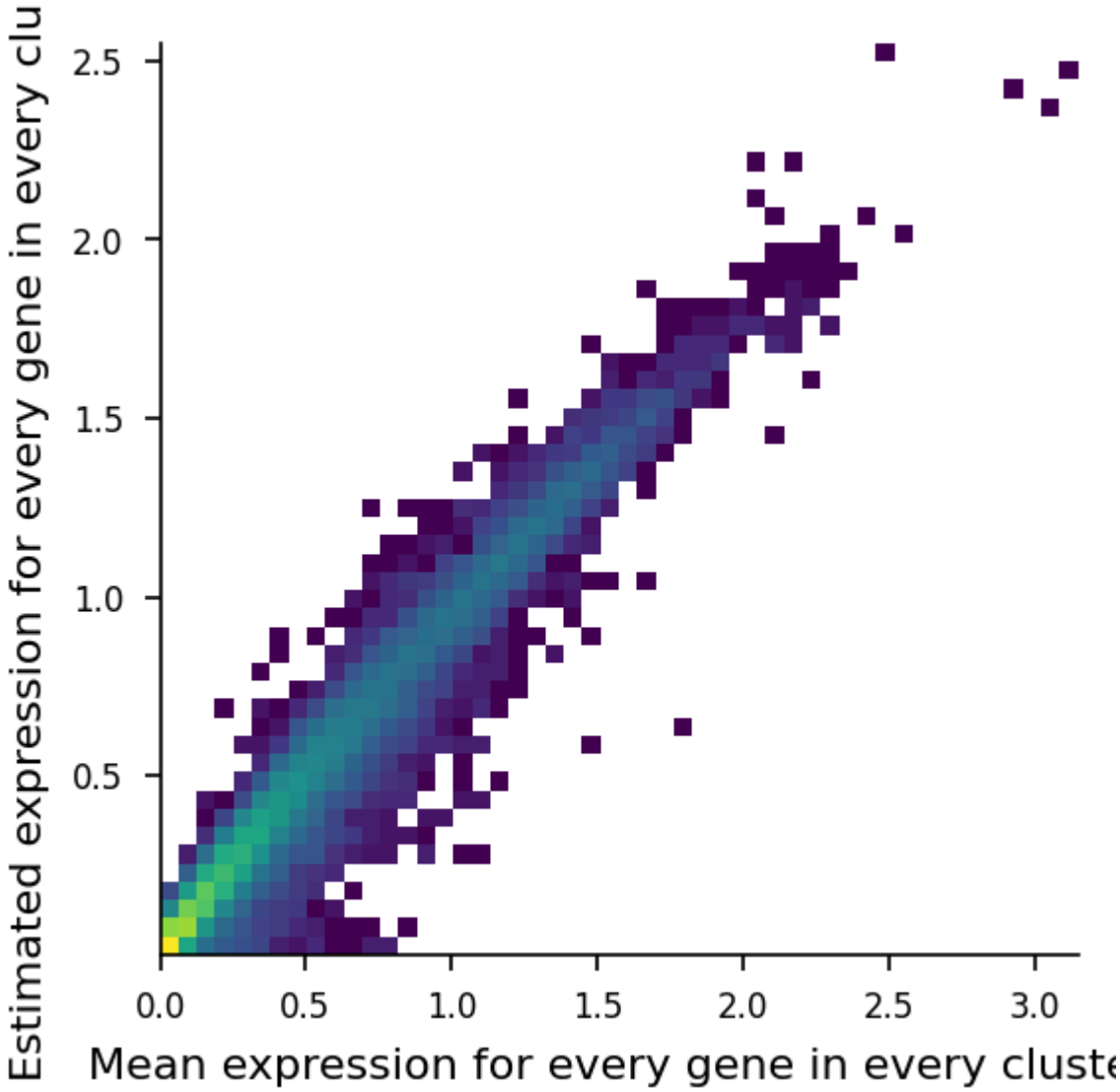
In [51]:
```
mod_reg
```

```
RegressionBackgroundDetectionTech model with the following params:
n_factors: 34
n_batch: 23
Training status: Trained
```

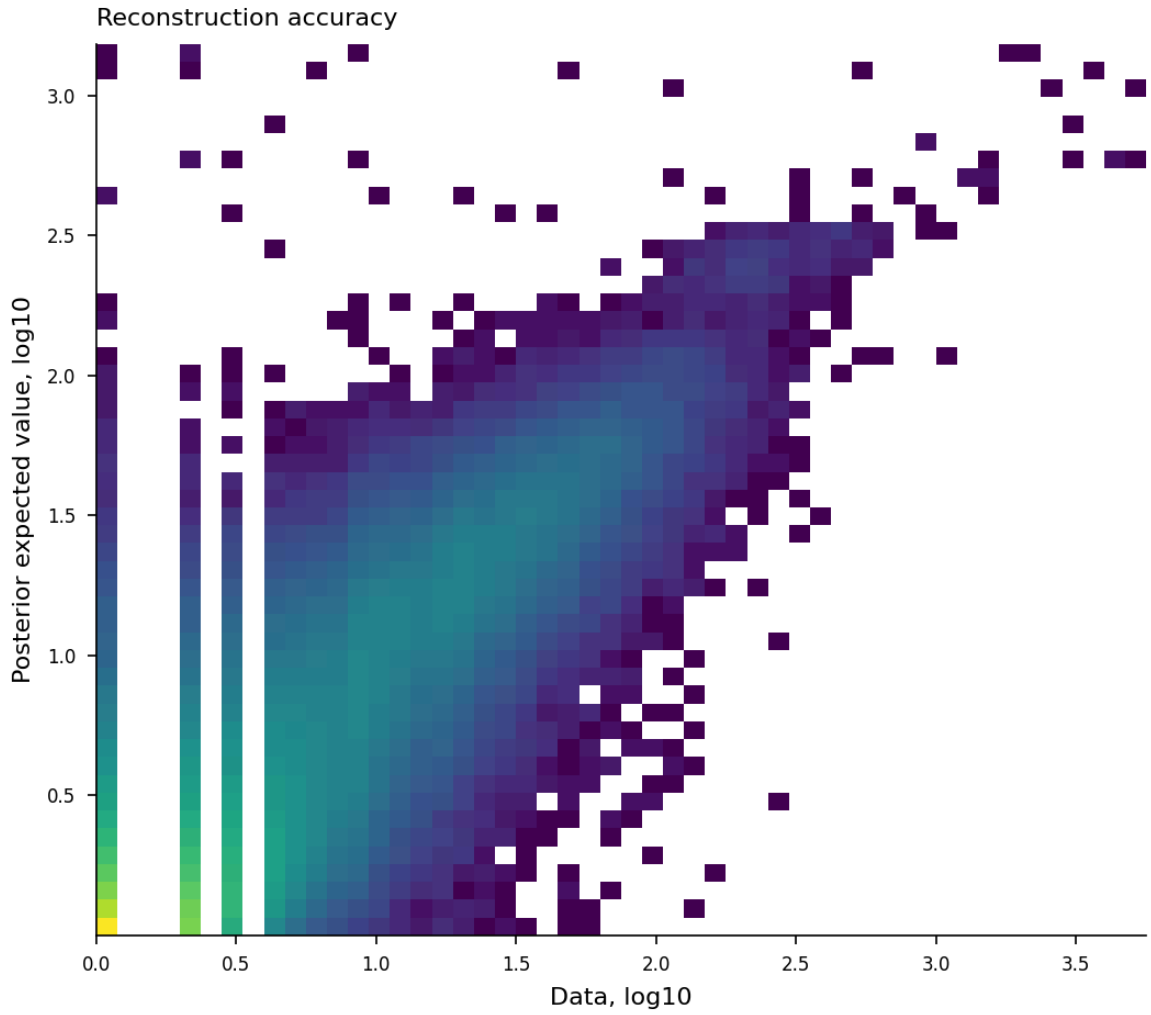
Out[51]:

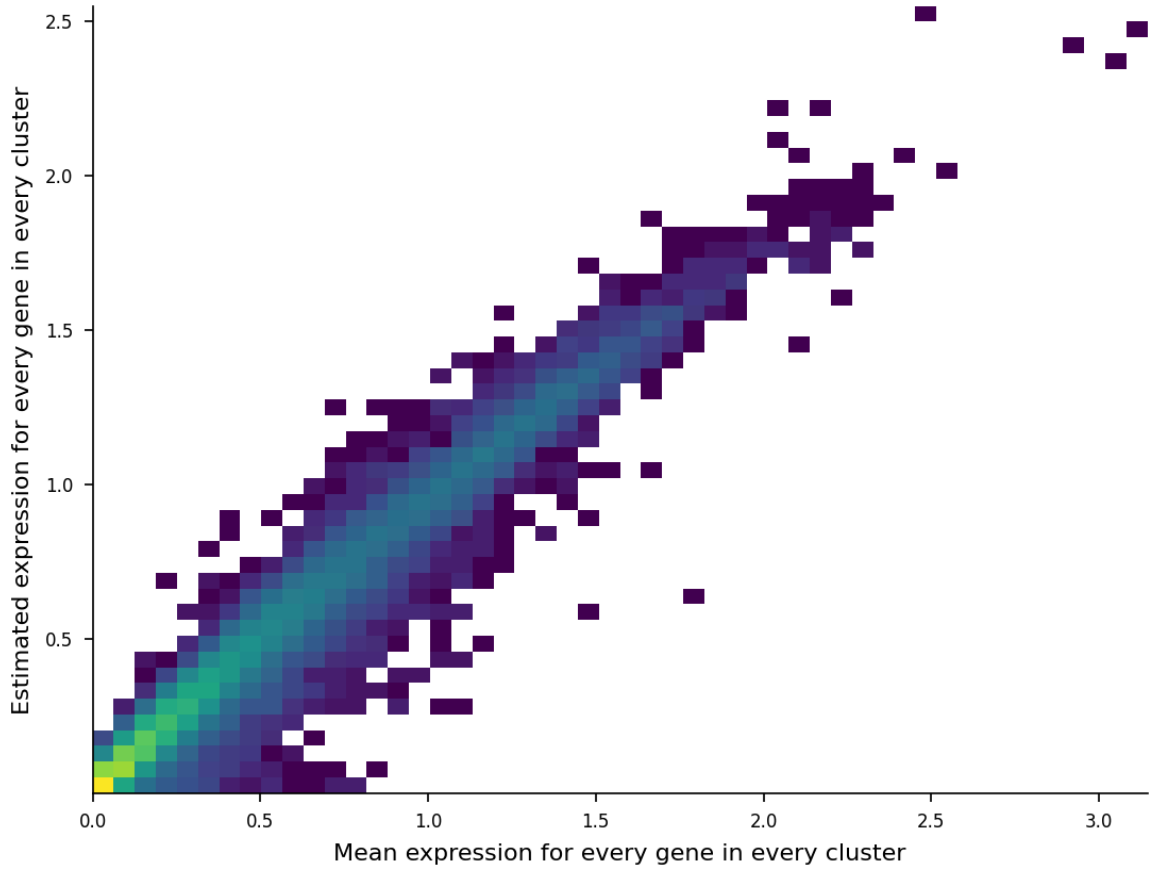
In [52]:
```
with plt.rc_context({"figure.figsize": (3,3)}):
    mod_reg.plot_QC(summary_name="q50")
```


Reconstruction accuracy
Posterior expected value, log10
Data, log10
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0



In [53]:
```
mod_reg.plot_QC(summary_name="q50")     # median signatures
plt.gcf().set_size_inches(2.5, 2.5)
plt.show()
```






```
<Figure size 500x500 with 0 Axes>
```

Extracting reference cell types signatures as a pd.DataFrame.
All parameters of the a Negative Binomial regression model are exported into reference anndata object, however for spatial mapping we just need the estimated expression of every gene in every cell type. Here we extract that from standard output:

In [54]:
```
[x for x in adata_ref.var.columns if 'cluster' in x]
```

Out[54]:
```
[]
```

In [55]:
```
# export estimated expression in each cluster
# (the original hard-coded 'means_per_cluster_mu_fg' -> KeyError after a
inf_aver, summary = get_reference_signatures(adata_ref, prefer=("means",
print(f"Using {summary} signatures")
inf_aver.iloc[0:5, 0:5]
```

```
Using q50 signatures
```

Out[55]:

GeneID-2	B_Cycling	B_GC_DZ	B_GC_LZ	B_GC_prePB	B_IFN
ENSG00000188976	0.433266	0.244424	0.311102	0.336622	0.150344
ENSG00000188290	0.001882	0.000460	0.000511	0.037657	0.037758
ENSG00000187608	0.384278	0.213292	0.274602	0.491036	3.888645
ENSG00000186891	0.019857	0.000512	0.055829	0.058270	0.006450
ENSG00000186827	0.007621	0.000338	0.005983	0.020730	0.008931

In [ ]:
```
```

Cell2location: spatial mapping

Find shared genes and prepare anndata. Subset both anndata and reference signatures:

In [56]:
```
# find shared genes and subset both anndata and reference signatures
intersect = np.intersect1d(adata_vis.var_names, inf_aver.index)
adata_vis = adata_vis[:, intersect].copy()
inf_aver = inf_aver.loc[intersect, :].copy()

len(intersect), len(adata_vis.var_names), len(inf_aver.index)
```

Out[56]:
```
(10217, 10217, 10217)
```

In [ ]:
```
```

cell2location

prepare anndata for cell2location model

In [57]:
```
cell2location.models.Cell2location.setup_anndata(adata=adata_vis, batch_k
```

Important
To use cell2location spatial mapping model, you need to specify 2 user-provided hyperparameters (**N_cells_per_location** and **detection_alpha**)

For detailed guidance on setting these hyperparameters and their impact see the flow diagram and the note.

Choosing hyperparameter `N_cells_per_location`!

It is useful to adapt the expected cell abundance `N_cells_per_location` to every tissue. This value can be estimated from paired histology images and as described in the note above. Change the value presented in this tutorial (`N_cells_per_location=30`) to the value observed in your your tissue.

Choosing hyperparameter `detection_alpha`!

To improve accuracy & sensitivity on datasets with large technical variability in RNA detection sensitivity within the slide/batch - you need to relax regularisation of per-location normalisation (use `detection_alpha=20`). High technical variability in RNA detection sensitivity is present in your sample when you observe the spatial distribution of total RNA count per location that doesn't match expected cell numbers based on histological examination.

We initially opted for high regularisation (`detection_alpha=200`) as a default because the mouse brain & human lymph node datasets used in our paper have low technical effects and using high regularisation strenght improves consistency between total estimated cell abundance per location and the nuclei count quantified from histology (Fig S8F in cell2location paper). However, in many collaborations, we see that Visium experiments on human tissues suffer from technical effects. This motivates the new default value of `detection_alpha=20` and the recommendation of testing both settings on your data (`detection_alpha=20` and `detection_alpha=200`).

In [58]:
```
# create and train the model
mod_c2l = cell2location.models.Cell2location(
    adata_vis, cell_state_df=inf_aver,
    # the expected average cell abundance: tissue-dependent
    # hyper-prior which can be estimated from paired histology:
    N_cells_per_location=30,
    # hyperparameter controlling normalisation of
    # within-experiment variation in RNA detection:
    detection_alpha=20
)
mod_c2l
```

```
cell2location model with the following params:
n_factors: 34
n_batch: 1
Training status: Not Trained
```

Out[58]:

In [59]:
```
mod_c2l.view_anndata_setup()
```

Anndata setup with scvi-tools version 1.5.1.

Setup via `Cell2location.setup_anndata` with arguments:

```
{
    'layer': None,
    'batch_key': 'sample',
    'labels_key': None,
    'categorical_covariate_keys': None,
    'continuous_covariate_keys': None
}
```

Summary Statistics

Summary Stat Key	Value
n_batch	1
n_cells	4035
n_extra_categorical_covs	0
n_extra_continuous_covs	0
n_labels	1
n_vars	10217

Data Registry

Registry Key	scvi-tools Location
X	adata.X
batch	adata.obs['_scvi_batch']
ind_x	adata.obs['_indices']
labels	adata.obs['_scvi_labels']

batch State Registry

Source Location	Categories	scvi-tools Encoding
adata.obs['sample']	V1_Human_Lymph_Node	0

labels State Registry

Source Location	Categories	scvi-tools Encoding
adata.obs['_scvi_labels']	0	0

The model and output h5ad can be loaded later like this:

```
adata_ref_filename = f"{run_name}/sp.h5ad"
adata_vis = sc.read_h5ad(adata_ref_filename)
mod_c2l = cell2location.models.Cell2location.load(f"{run_name}",
adata_vis)
```

In [60]:
```
adata_file_sp = run_name / "sp.h5ad"
adata_file_sp.exists(), adata_file_sp
```

Out[60]:
```
(True,
 PosixPath('/home/flavio/uv/squidpy/results/lymph_nodes_analysis/cell2location_map/sp.h5ad'))
```

Training cell2location

and

Exporting estimated posterior distributions of cell abundance and saving results

In [61]:
```
if not adata_file_sp.exists():
    mod_c2l.train(max_epochs=15000,
            # train using full data (batch_size=None)
            batch_size=None,
            # use all data points in training because
            # we need to estimate cell abundance at all locations
            train_size=1,
            accelerator='gpu',
            )

    # In this section, we export the estimated cell abundance (summary of
    adata_vis = mod_c2l.export_posterior(
        adata_vis, use_quantiles=True,
        add_to_obsm=["q05", "q50", "q95", "q0001"],
        sample_kwargs={'batch_size': mod_c2l.adata.n_obs, 'accelerator':
        # sample_kwargs={'num_samples': 1000, 'batch_size': mod_c2l.adata
    )

    # Compute expected expression per cell type
    expected_dict = mod_c2l.module.model.compute_expected_per_cell_type(
        mod_c2l.samples["post_sample_q05"], mod_c2l.adata_manager
    )

    # Add to anndata layers
    for i, n in enumerate(mod_c2l.factor_names_):
        adata_vis.layers[n] = expected_dict['mu'][i]

    # Save model
    mod_c2l.save(f"{run_name}", overwrite=True)
    print(f"Model saved to {run_name}")

    # Save anndata object with results
    adata_vis.write(adata_file_sp)
    print(f"adata_vis saved to {adata_file_sp}")
else:
    # Load AnnData with posterior abundances in .obsm (already registered
    adata_vis = sc.read_h5ad(adata_file_sp)
    print(f"adata_vis read from {adata_file_sp}")

    # Reload the trained spatial model bound to that AnnData
    mod_c2l = cell2location.models.Cell2location.load(f"{run_name}", adat
    print(f"Model loaded from {run_name}")

    # mod_c2l.samples is not saved with the model: rebuild it (needed by
    adata_vis = ensure_posterior_samples(mod_c2l, adata_vis, quantiles=["
    print("posterior samples:", sorted(mod_c2l.samples.keys()))
```

```
# Common to both branches: copy 5% quantile abundances into .obs for plot
factor_names = adata_vis.uns['mod']['factor_names']
adata_vis.obs[factor_names] = adata_vis.obsm['q05_cell_abundance_w_sf']
```

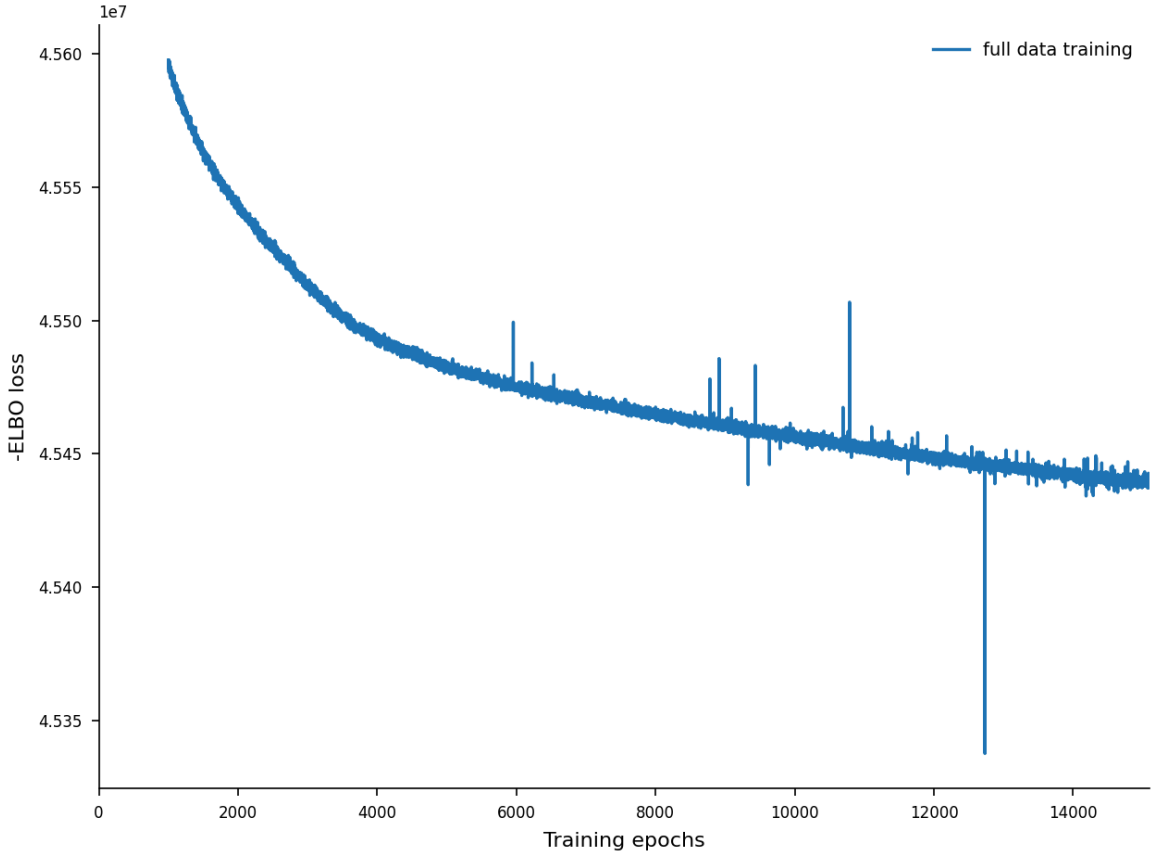
```
adata_vis read from /home/flavio/uv/squidpy/results/lymph_nodes_analysis/cell2location_map/sp.h5ad
INFO     File /home/flavio/uv/squidpy/results/lymph_nodes_analysis/cell2location_map/model.pt already downloaded
Model loaded from /home/flavio/uv/squidpy/results/lymph_nodes_analysis/cell2location_map
posterior samples: ['post_sample_q0001', 'post_sample_q05', 'post_sample_q50', 'post_sample_q95']
```

In [62]:
```
mod_c2l.adata.n_obs
```

Out[62]: 4035

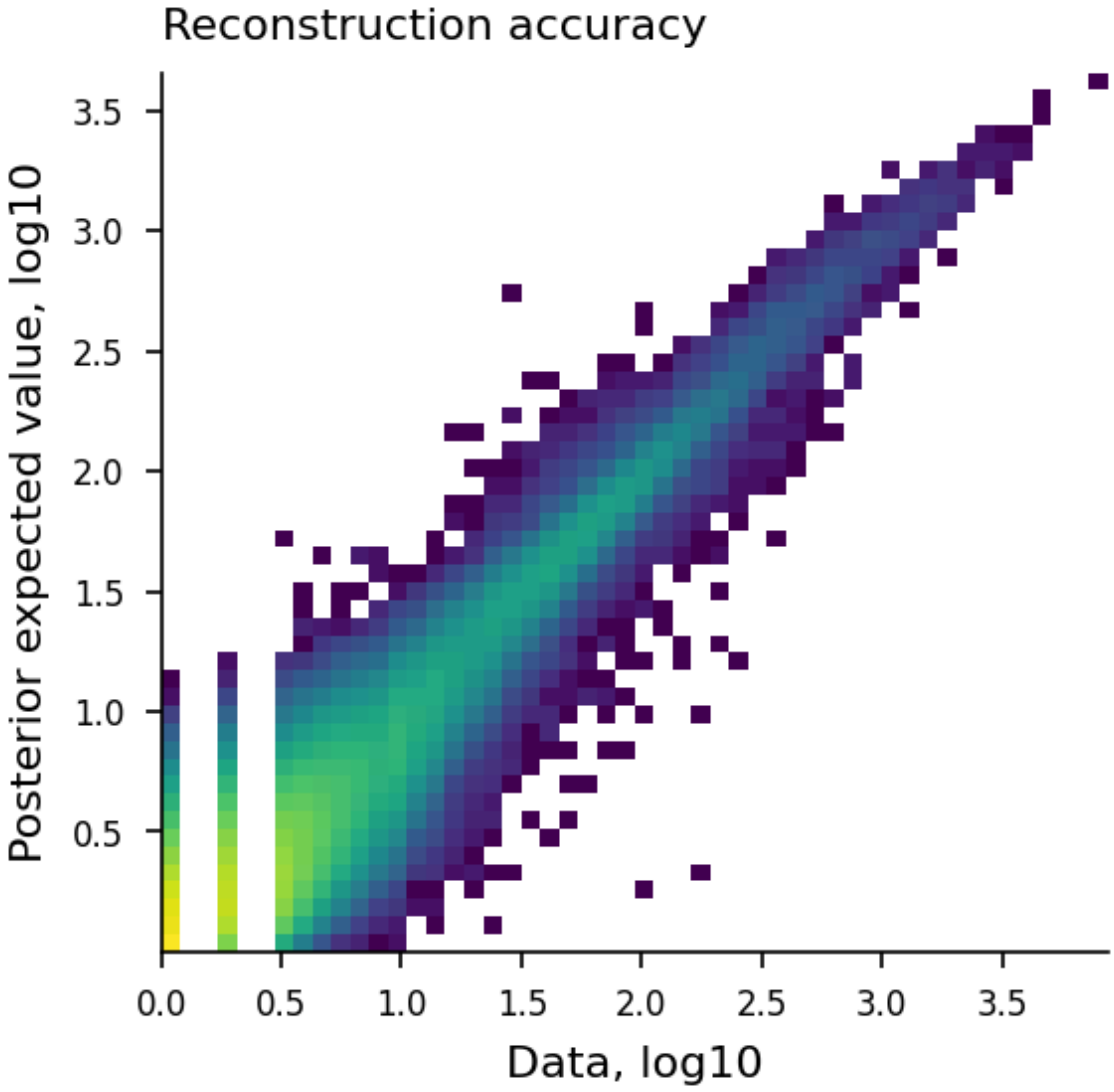
plot ELBO loss history during training, removing first 100 epochs from the plot

In [63]:
```
mod_c2l.plot_history(1000)
plt.legend(labels=['full data training']);
```



Assessing mapping quality. Examine reconstruction accuracy to assess if there are any issues with mapping. The plot should be roughly diagonal, strong deviations will signal problems that need to be investigated.

In [64]:
```
with plt.rc_context({"figure.figsize": (3,3)}):
    mod_c2l.plot_QC(summary_name="q50")    # median signatures
```



When intergrating multiple spatial batches and when working with datasets that have substantial variation of detected RNA within slides (that cannot be explained by high cellular density in the histology), it is important to assess whether cell2location normalised those effects. You expect to see similar total cell abundance across batches but distinct RNA detection sensitivity (both estimated by cell2location). You expect total cell abundance to mirror high cellular density in the histology.

```
fig = mod_c2l.plot_spatial_QC_across_batches()
```

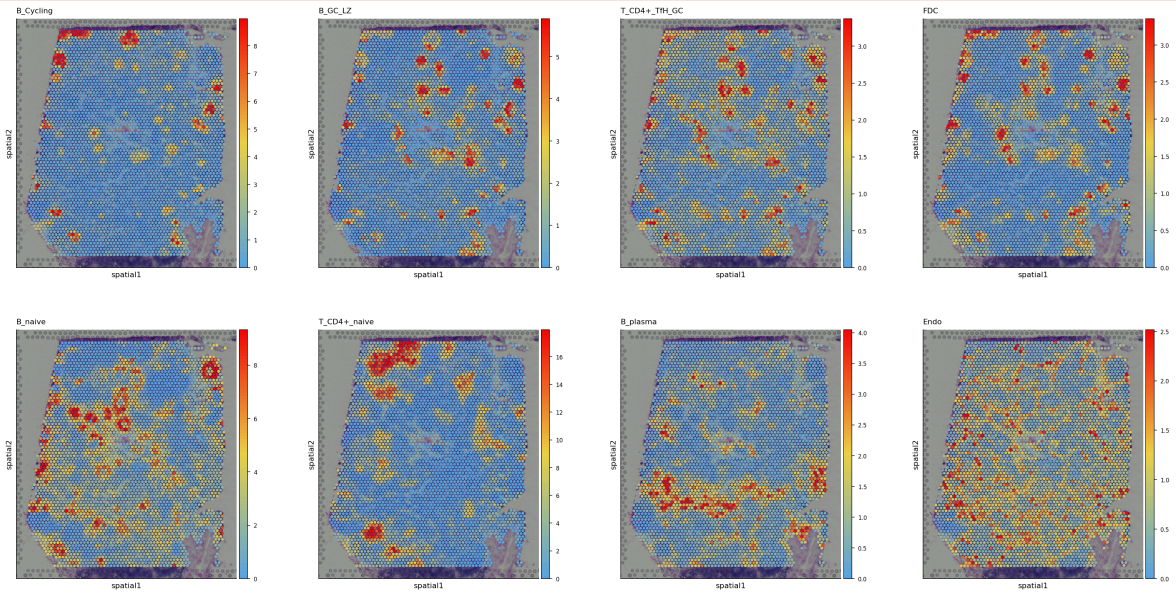
Visualising cell abundance in spatial coordinates

> Note
> We use 5% quantile of the posterior distribution, representing the value of cell abundance that the model has high confidence in (aka 'at least this amount is present').

In [65]:
```
# add 5% quantile, representing confident cell abundance, 'at least this
# to adata.obs with nice names for plotting
adata_vis.obs[adata_vis.uns['mod']['factor_names']]= adata_vis.obsm['q05_
```

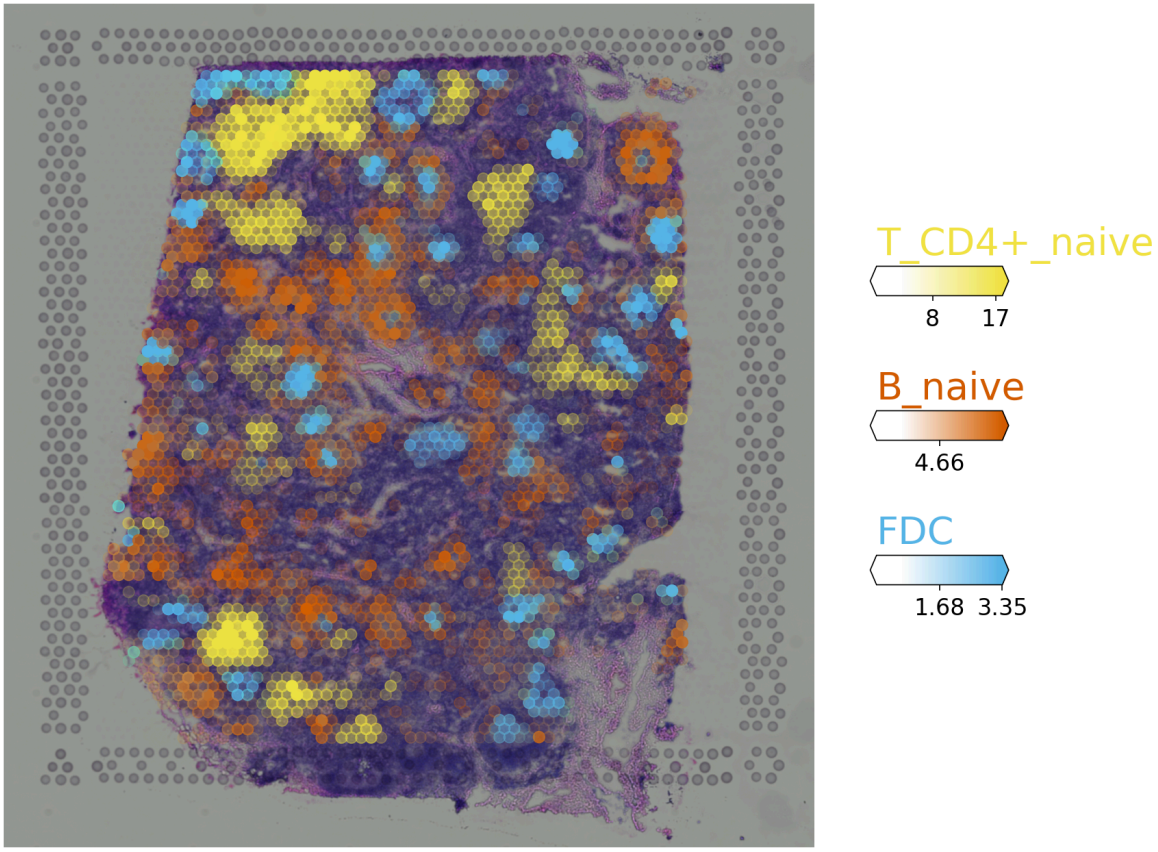
```
# select one slide
from cell2location.utils import select_slide
slide = select_slide(adata_vis, 'V1_Human_Lymph_Node')

# plot in spatial coordinates
with mpl.rc_context({'axes.facecolor':  'black',
                     'figure.figsize': [4.5, 5]}):

    sc.pl.spatial(slide, cmap=cmap_byr,
                  # show first 8 cell types
                  color=['B_Cycling', 'B_GC_LZ', 'T_CD4+_TfH_GC', 'FDC',
                         'B_naive', 'T_CD4+_naive', 'B_plasma', 'Endo'],
                  ncols=4, size=1.3,
                  img_key='hires',
                  # limit color scale at 99.2% quantile of cell abundance
                  vmin=0, vmax='p99.2'
                 )
```

```
/tmp/ipykernel_416266/1310335525.py:13: FutureWarning: The function spatia
l is deprecated and will be removed in the future. Use :func:`squidpy.pl.s
patial_scatter` instead.
  sc.pl.spatial(slide, cmap=cmap_byr,
```



In [66]:
```
# Now we use cell2location plotter that allows showing multiple cell type
from cell2location.plt import plot_spatial

# select up to 6 clusters
# Follicular dendritic cells (FDCs)
clust_labels = ['T_CD4+_naive', 'B_naive', 'FDC']
clust_col = [str(i) for i in clust_labels]# in case column names differ f

slide = select_slide(adata_vis, 'V1_Human_Lymph_Node')

with mpl.rc_context({'figure.figsize': (10,8)}):
    fig = plot_spatial(
        adata=slide,
        # labels to show on a plot
        color=clust_col, labels=clust_labels,
        show_img=True,
        # 'fast' (white background) or 'dark_background'
        style='fast',
```

```
        # limit color scale at 99.2% quantile of cell abundance
        max_color_quantile=0.992,
        # size of locations (adjust depending on figure size)
        circle_diameter=6,
        colorbar_position='right'
    )
```



In [ ]:

Downstream analysis

Identifying discrete tissue regions by Leiden clustering<a class="anchor"

id="Identifying-discrete-tissue-regions-by-Leiden-clustering">

We identify tissue regions that differ in their cell composition by clustering locations using cell abundance estimated by cell2location.

We find tissue regions by clustering Visium spots using estimated cell abundance each cell type. We constuct a K-nearest neigbour (KNN) graph representing similarity of locations in estimated cell abundance and then apply Leiden clustering. The number of KNN neighbours should be adapted to size of dataset and the size of anatomically defined regions (e.i. hippocampus regions are rather small compared to size of the brain so could be masked by large `n_neighbors`). This can be done for a range KNN neighbours and Leiden clustering resolutions until a clustering matching the anatomical structure of the tissue is obtained.

The clustering is done jointly across all Visium sections / batches, hence the region identities are directly comparable. When there are strong technical effects between multiple batches (not the case here) `sc.external.pp.bbknn` can be in principle used to account for those effects during the KNN construction.

The resulting clusters are saved in `adata_vis.obs['region_cluster']`.

In [67]:
```
# compute KNN using the cell2location output stored in adata.obsm
sc.pp.neighbors(adata_vis, use_rep='q05_cell_abundance_w_sf', n_neighbors

# Cluster spots into regions using scanpy
sc.tl.leiden(adata_vis, resolution=1.1)

# add region as categorical variable
adata_vis.obs["region_cluster"]= adata_vis.obs["leiden"].astype("category
```

```
/tmp/ipykernel_416266/2927696082.py:5: FutureWarning: The `igraph` impleme
ntation of leiden clustering is *orders of magnitude faster*. Set the flav
or argument to (and install if needed) 'igraph' to use it.
In the future, the default backend for leiden will be igraph instead of le
idenalg. To achieve the future defaults please pass: `flavor='igraph'` and
`n_iterations=2`. `directed` must also be `False` to work with igraph's im
plementation.
  sc.tl.leiden(adata_vis, resolution=1.1)
```

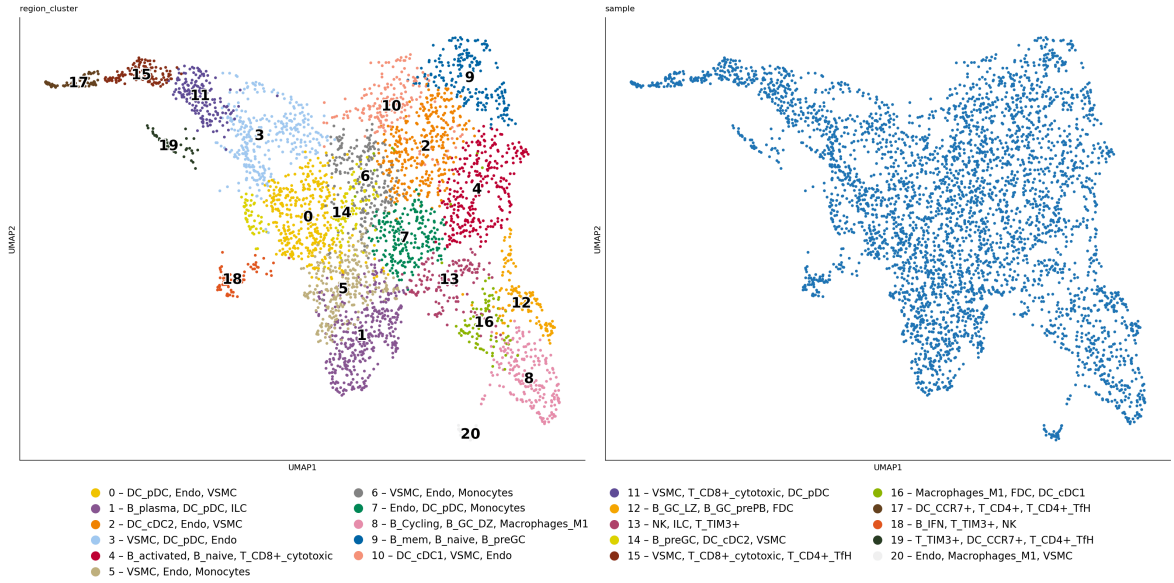
We can use the location composition similarity graph to build a joint integrated UMAP representation of all section/Visium batches.

In [68]:
```
ab = adata_vis.obsm['q05_cell_abundance_w_sf'].copy()
ab.columns = [c.replace('q05cell_abundance_w_sf_', '') for c in ab.column
ab['region_cluster'] = adata_vis.obs['region_cluster'].values

# mean abundance per cluster, scaled per cell type
mean_ab = ab.groupby('region_cluster', observed=True).mean()
mean_ab_scaled = mean_ab / mean_ab.max(axis=0)

# top 3 cell types per cluster
top = mean_ab_scaled.apply(lambda r: ', '.join(r.nlargest(3).index), axis

names = {i: top.iloc[i] for i in range(len(top))}
names
```

Out[68]:
```
{0: 'DC_pDC, Endo, VSMC',
 1: 'B_plasma, DC_pDC, ILC',
 2: 'DC_cDC2, Endo, VSMC',
 3: 'VSMC, DC_pDC, Endo',
 4: 'B_activated, B_naive, T_CD8+_cytotoxic',
 5: 'VSMC, Endo, Monocytes',
 6: 'VSMC, Endo, Monocytes',
 7: 'Endo, DC_pDC, Monocytes',
 8: 'B_Cycling, B_GC_DZ, Macrophages_M1',
 9: 'B_mem, B_naive, B_preGC',
 10: 'DC_cDC1, VSMC, Endo',
 11: 'VSMC, T_CD8+_cytotoxic, DC_pDC',
 12: 'B_GC_LZ, B_GC_prePB, FDC',
 13: 'NK, ILC, T_TIM3+',
 14: 'B_preGC, DC_cDC2, VSMC',
 15: 'VSMC, T_CD8+_cytotoxic, T_CD4+_TfH',
 16: 'Macrophages_M1, FDC, DC_cDC1',
 17: 'DC_CCR7+, T_CD4+, T_CD4+_TfH',
 18: 'B_IFN, T_TIM3+, NK',
 19: 'T_TIM3+, DC_CCR7+, T_CD4+_TfH',
 20: 'Endo, Macrophages_M1, VSMC'}
```

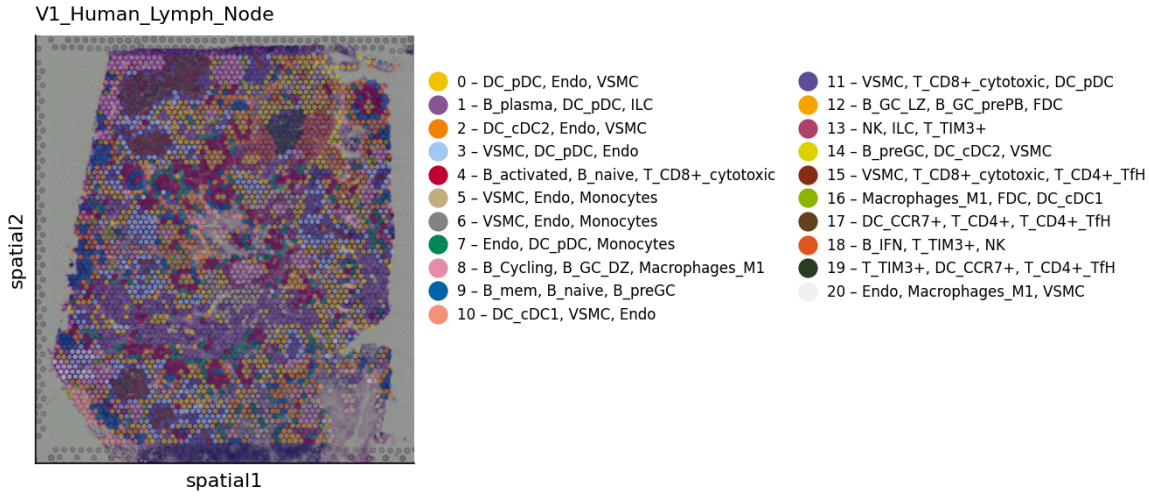
In [69]:
```
adata_vis.obs['region_name'] = (
    adata_vis.obs['region_cluster'].astype(str).map(names).fillna('unassi
)
```

In [70]:
```
# names: your manual dict, or fall back to the top cell types per cluster
# names = {'0': 'B follicle', '1': 'T zone', ...}
names = top.to_dict()        # from the previous step: cluster -> "type1,

with mpl.rc_context({'axes.facecolor': 'white'}):
    fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(16, 8))

    # numbers written on each region
    sc.pl.umap(adata_vis, color='region_cluster', size=30,
               legend_loc='on data', legend_fontsize=14,
               legend_fontoutline=2,          # white outline, readable o
               ax=axes[0], show=False, title='region_cluster')

    sc.pl.umap(adata_vis, color='sample', size=30,
               legend_loc=None, ax=axes[1], show=False)

    # bottom legend: color + "number — name"
    cats   = adata_vis.obs['region_cluster'].cat.categories
    colors = adata_vis.uns['region_cluster_colors']
    handles = [Line2D([0], [0], marker='o', linestyle='', markersize=9,
                      markerfacecolor=c, markeredgecolor='none',
                      label=f"{s} — {names.get(s, '')}")
               for s, c in zip(cats, colors)]

    fig.legend(handles=handles, loc='lower center', ncol=4,
               fontsize=10, frameon=False, bbox_to_anchor=(0.5, 0.0),
               handletextpad=0.3, columnspacing=1.5)

    plt.tight_layout(rect=[0, 0.18, 1, 1])   # more room: labels are long
    plt.show()
```



In [72]:
```
s = 'V1_Human_Lymph_Node'

with mpl.rc_context({'axes.facecolor': 'black'}):
    fig, ax = plt.subplots(figsize=(8,4))
    sc.pl.spatial(adata_vis, color='region_cluster', library_id=s,
                  size=1.3, img_key='hires', alpha=0.5,
                  legend_loc=None, ax=ax, show=False, title=s)

    cats   = adata_vis.obs['region_cluster'].cat.categories
    colors = adata_vis.uns['region_cluster_colors']
    handles = [Line2D([0], [0], marker='o', linestyle='', markersize=8,
                      markerfacecolor=c, markeredgecolor='none',
                      label=f"{k} – {names.get(k, '')}")
               for k, c in zip(cats, colors)]

    fig.legend(handles=handles, loc='lower center', ncol=2,
               fontsize=6, frameon=False, bbox_to_anchor=(0.9, 0.5),
               handletextpad=0.3, columnspacing=1.2)

    plt.tight_layout(rect=[0, 0.25, 1, 1])
    plt.show()
```

```
/tmp/ipykernel_416266/2873233873.py:5: FutureWarning: The function spatial
is deprecated and will be removed in the future. Use :func:`squidpy.pl.spa
tial_scatter` instead.
  sc.pl.spatial(adata_vis, color='region_cluster', library_id=s,
```



In [73]:
```
import plotly.graph_objects as go

kelly = ['#F3C300', '#875692', '#F38400', '#A1CAF1', '#BE0032', '#C2B280'
         '#848482', '#008856', '#E68FAC', '#0067A5', '#F99379', '#604E97'
         '#F6A600', '#B3446C', '#DCD300', '#882D17', '#8DB600', '#654522'
         '#E25822', '#2B3D26', '#F2F3F4', '#222222']

cats = adata_vis.obs['region_cluster'].cat.categories
colors = kelly[:len(cats)]
adata_vis.uns['region_cluster_colors'] = colors   # keeps scanpy/UMAP plo

s   = 'V1_Human_Lymph_Node'
lib = adata_vis.uns['spatial'][s]

# background image (hires) as uint8
img = lib['images']['hires']
if img.dtype != np.uint8:
    img = (np.clip(img, 0, 1) * 255).astype(np.uint8)

# spot coordinates scaled to the hires image
sf = lib['scalefactors']['tissue_hires_scalef']
xy = adata_vis.obsm['spatial'] * sf

cats   = adata_vis.obs['region_cluster'].cat.categories
colors = adata_vis.uns['region_cluster_colors']
clust  = adata_vis.obs['region_cluster'].astype(str).values

fig = go.Figure()
fig.add_trace(go.Image(z=img, hoverinfo='skip'))

for k, c in zip(cats, colors):
    m = clust == k
    label = f"{k} — {names.get(k, '')}"
    fig.add_trace(go.Scattergl(
        x=xy[m, 0], y=xy[m, 1],
        mode='markers',
        marker=dict(size=6, color=c, opacity=0.9, line=dict(width=0.5, co
        name=label,
        text=adata_vis.obs_names[m],
        hovertemplate=f"<b>{label}</b><br>spot: %{{text}}<extra></extra>"
    ))

fig.update_layout(
    title=s,
    width=1400, height=900,
    plot_bgcolor='black',
    margin=dict(l=10, r=10, t=40, b=10),
    legend=dict(font=dict(size=15), itemsizing='constant',
                x=1.01, y=0.5, yanchor='middle'),   # right side
)
fig.update_xaxes(visible=False, range=[0, img.shape[1]])
fig.update_yaxes(visible=False, range=[img.shape[0], 0],  # image coords:
                 scaleanchor='x')
fig.show()
```

Grouped the panel genes into five cell-type markers:

1. pan-B (CD19/MS4A1/CD79A/B/PAX5/CD22)

2. follicular mantle B (IGHD/IGHM)
3. pan-lymphocyte (CD52/CD53)
4. TIM3+ T, GC TfH, Treg, GC DZ B (CD27)
5. monocytes/macrophages (CD14)

CDS2 doesn't mark any clear cell type.

In [74]:
```
ct = adata_ref.obs["Subset"].astype(str).values          # cell-type labe

# found = genes present in the reference; X = raw counts; E = log1p(CP10k
found, X, E = panel_lognorm(adata_ref, genes_plot)

pct  = pd.DataFrame(X > 0, columns=found).groupby(ct).mean() * 100   # %
mean = pd.DataFrame(E,     columns=found).groupby(ct).mean()         # me
n_ct = pd.Series(ct).value_counts()                                  # ce

# row order for the dot plot: cell types grouped by lineage
lineage = {
    "GC B":        ["B_GC_DZ","B_GC_LZ","B_GC_prePB","B_Cycling","B_preGC
    "naive/mem B": ["B_naive","B_activated","B_IFN","B_mem"],
    "plasma":      ["B_plasma"],
    "T":           ["T_CD4+_naive","T_CD4+","T_CD4+_TfH","T_CD4+_TfH_GC",
                    "T_CD8+_naive","T_CD8+_cytotoxic","T_CD8+_CD161+"],
    "NK/ILC":      ["NK","NKT","ILC"],
    "myeloid":     ["Monocytes","Macrophages_M1","Macrophages_M2","DC_cDC
    "stromal":     ["FDC","Endo","VSMC"],
}
ct_order = [c for v in lineage.values() for c in v]
assert set(ct_order) == set(ct), set(ct) ^ set(ct_order)

P = pct.loc[ct_order, found]
M = mean.loc[ct_order, found]
U=adata_ref.obsm["X_umap"]

Q   = pd.DataFrame(np.asarray(adata_vis.obsm["q05_cell_abundance_w_sf"]),
                   index=adata_vis.obs_names, columns=list(adata_vis.uns[
```

In [75]:
```
lib = list(adata_vis.uns["spatial"].keys())[0]                       # 'V

img = adata_vis.uns["spatial"][lib]["images"]["hires"]
sf  = adata_vis.uns["spatial"][lib]["scalefactors"]["tissue_hires_scalef"
xy  = adata_vis.obsm["spatial"] * sf                                 # sp

np.unique(ct)
```

Out[75]:
```
array(['B_Cycling', 'B_GC_DZ', 'B_GC_LZ', 'B_GC_prePB', 'B_IFN',
       'B_activated', 'B_mem', 'B_naive', 'B_plasma', 'B_preGC',
       'DC_CCR7+', 'DC_cDC1', 'DC_cDC2', 'DC_pDC', 'Endo', 'FDC', 'ILC',
       'Macrophages_M1', 'Macrophages_M2', 'Mast', 'Monocytes', 'NK',
       'NKT', 'T_CD4+', 'T_CD4+_TfH', 'T_CD4+_TfH_GC', 'T_CD4+_naive',
       'T_CD8+_CD161+', 'T_CD8+_cytotoxic', 'T_CD8+_naive', 'T_TIM3+',
       'T_TfR', 'T_Treg', 'VSMC'], dtype=object)
```

In [76]:
```
cmaps={"GC B":"Reds","naive/mem B":"Blues","plasma":"Purples","T":"Greens
order=[c for v in lineage.values() for c in v]; assert set(order)==set(ct
col={}
for k,v in lineage.items():
    cm=plt.get_cmap(cmaps[k]);
```

```
    for j,c in enumerate(v): col[c]=cm(0.45+0.45*(j/(max(len(v)-1,1))))

focus=["B_GC_LZ","B_GC_DZ","B_Cycling","B_preGC","B_naive","B_mem","B_pla
```

Identifying cellular compartments / tissue zones using matrix factorisation (NMF)

Here, we use the cell2location mapping results to identify the spatial co-occurrence of cell types in order to better understand the tissue organisation and predict cellular interactions. We performed non-negative matrix factorization (NMF) of the cell type abundance estimates from cell2location (paper section 4, Fig 4D). Similar to the established benefits of applying NMF to conventional scRNA-seq, the additive NMF decomposition yielded a grouping of spatial cell type abundance profiles into components that capture co-localised cell types (Supplemenary Methods section 4.2, p. 60). This NMF-based decomposition naturally accounts for the fact that multiple cell types and microenvironments can co-exist at the same Visium locations (see paper Fig S20, p. 34), while sharing information across tissue areas (e.g. individual germinal centres).

Tip

In practice, it is better to train NMF for a range of factors $R = 5, .., 30$ and select R as a balance between capturing fine-grained and splitting known well-established tissue zones.

If you want to find a few most disctinct cellular compartments, use a small number of factors.
If you want to find very strong co-location signal and assume that most cell types don't co-locate, use a lot of factors (> 30 - used here).

Below we show how to perform this analysis. To aid this analysis, we wrapped the analysis shown the notebook on advanced downstream analysis into a pipeline that automates training of the NMF model with varying number of factors:

In [77]:
```
fname = "res_dict_coloc.pkl"
filename = root_results / fname

filename.exists()
```

Out[77]: True

In [78]:
```
from cell2location import run_colocation
import pickle

fname = "res_dict_coloc.pkl"
filename = root_results / fname

if filename.exists():
    with open(filename, "rb") as fh:
        res_dict = pickle.load(fh)
        print(f"Results loaded from {filename}")
```

```
else:
    res_dict, adata_vis = run_colocation(
        adata_vis,
        model_name='CoLocatedGroupsSklearnNMF',
        train_args={
          'n_fact': np.arange(11, 13), # IMPORTANT: use a wider range of
          'sample_name_col': 'sample', # columns in adata_vis.obs that id
          'n_restarts': 3 # number of training restarts
        },
        # the hyperparameters of NMF can be also adjusted:
        model_kwargs={'alpha': 0.01, 'init': 'random', "nmf_kwd_args": {"
        export_args={'path': f'{run_name}/CoLocatedComb/'}
    )

    with open(filename, "wb") as fh:
      pickle.dump(res_dict, fh)
      print(f"Results saved to {filename}")
```

```
Results loaded from /home/flavio/uv/squidpy/results/res_dict_coloc.pkl
```

For every factor number, the model produces the following list of folder outputs:

`cell_type_fractions_heatmap/` : a dot plot of the estimated NMF weights of cell types (rows) across NMF components (columns)

`cell_type_fractions_mean/` : the data used for dot plot

`factor_markers/` : tables listing top 10 cell types most speficic to each NMF factor

`models/` : saved NMF models

`predictive_accuracy/` : 2D histogram plot showing how well NMF explains cell2location output

`spatial/` : NMF weights across locatinos in spatial coordinates

`location_factors_mean/` : the data used for the plot in spatial coordiantes

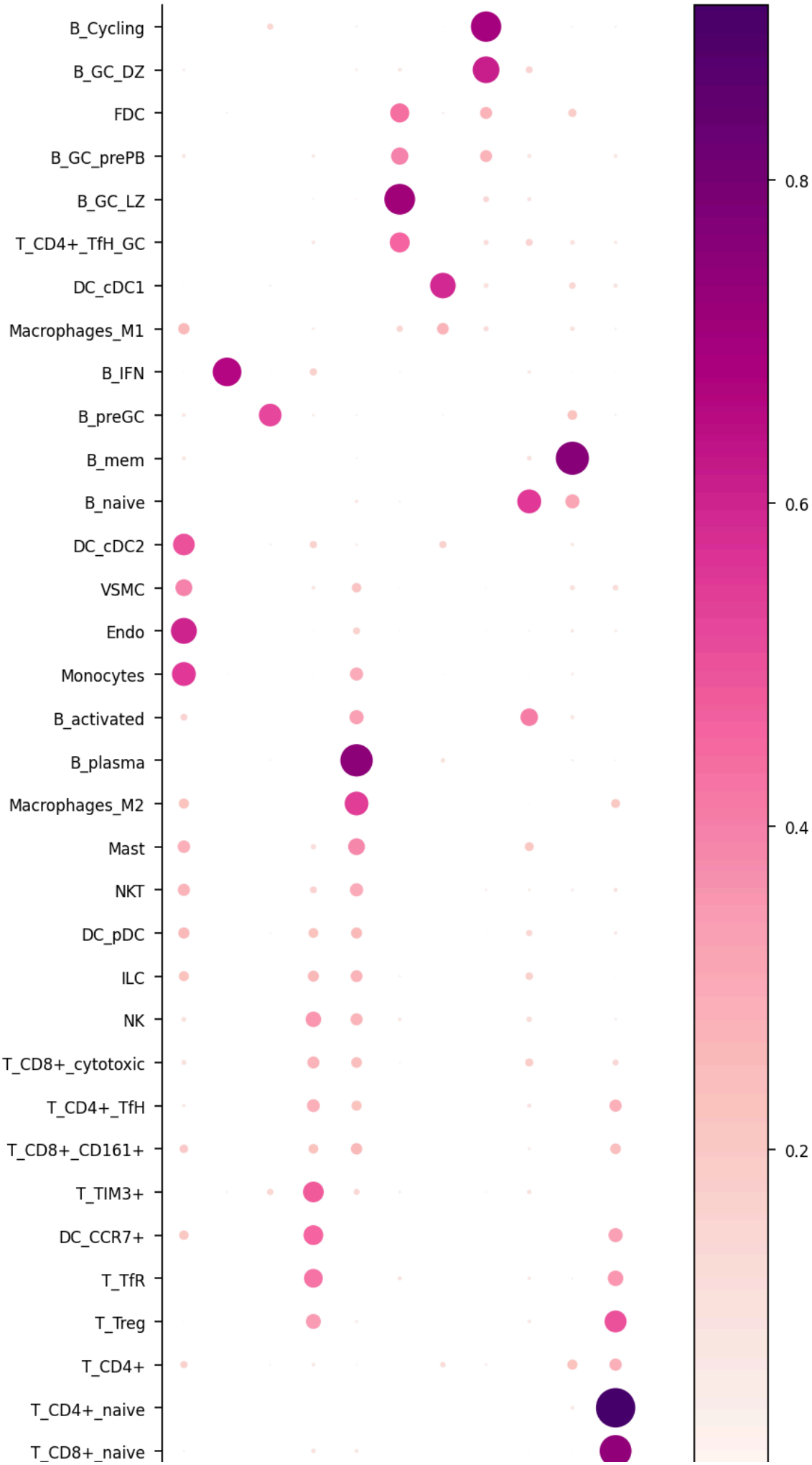
`stability_plots/` : stability of NMF weights between training restarts

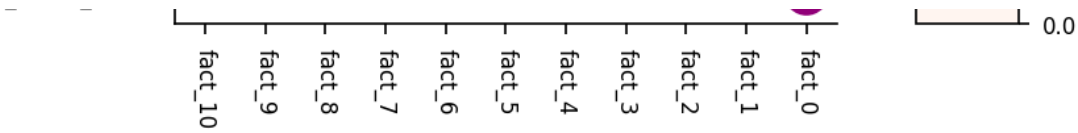
Key output that you want to examine are the files in `cell_type_fractions_heatmap/` which show a dot plot of the estimated NMF weights of cell types (rows) across NMF components (columns) which correspond to cellular compartments. Shown are relative weights, normalized across components for every cell type.

Tip

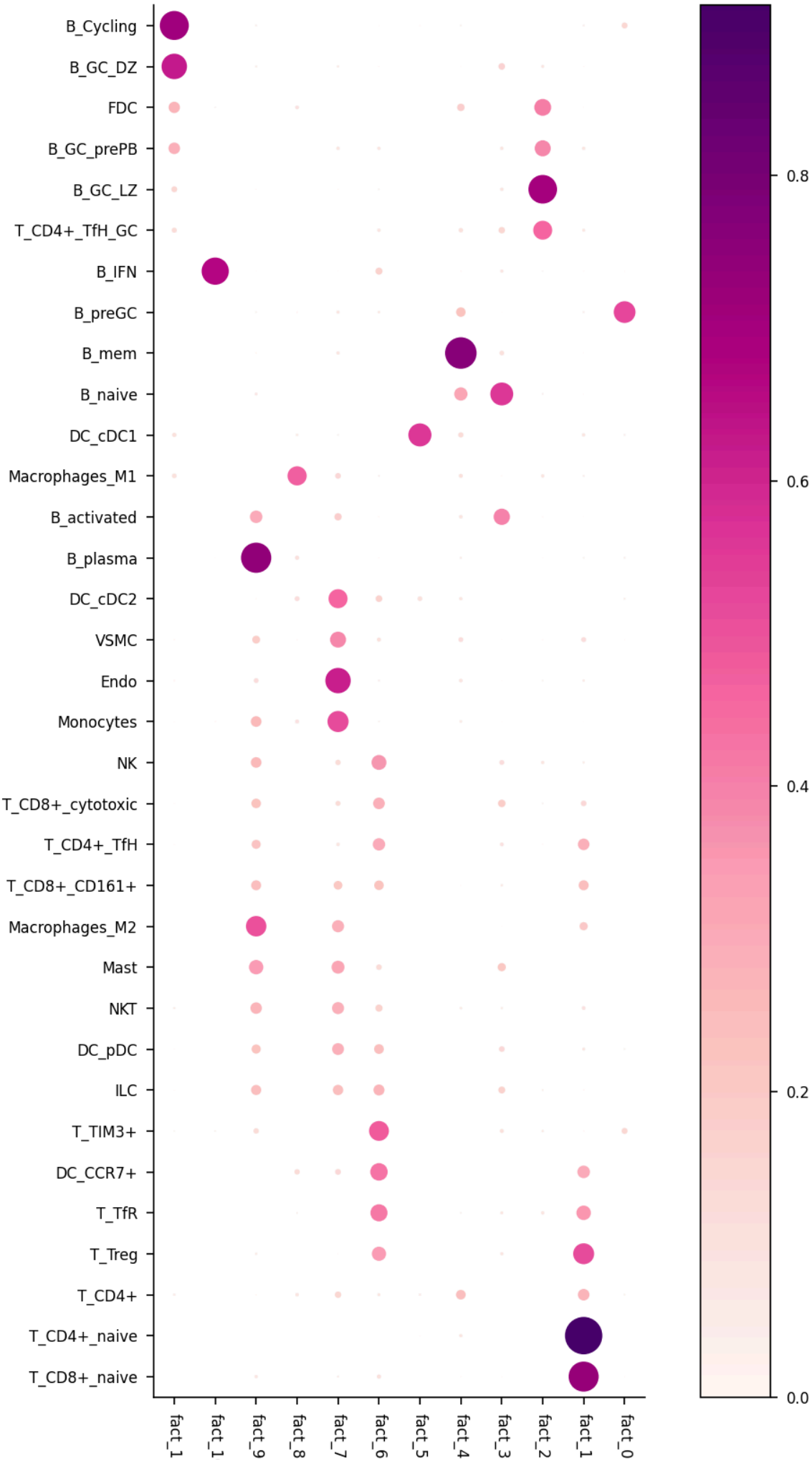
The NMF model output such as factor loadings are stored in `adata.uns[f"mod_coloc_n_fact{n_fact}"]` in a similar output format as main cell2location results in `adata.uns['mod']` .

In [79]:
```
res_dict['n_fact11']['mod'].plot_cell_type_loadings()
```


B_Cycling
B_GC_DZ
FDC
B_GC_prePB
B_GC_LZ
T_CD4+_TfH_GC
DC_cDC1
Macrophages_M1
B_IFN
B_preGC
B_mem
B_naive
DC_cDC2
VSMC
Endo
Monocytes
B_activated
B_plasma
Macrophages_M2
Mast
NKT
DC_pDC
ILC
NK
T_CD8+_cytotoxic
T_CD4+_TfH
T_CD8+_CD161+
T_TIM3+
DC_CCR7+
T_TfR
T_Treg
T_CD4+
T_CD4+_naive
T_CD8+_naive
0.8
0.6
0.4
0.2



In [80]:
```
# Here we plot the NMF weights (Same as saved to `cell_type_fractions_hea
res_dict['n_fact12']['mod'].plot_cell_type_loadings()
```


B_Cycling
B_GC_DZ
FDC
B_GC_prePB
B_GC_LZ
T_CD4+_TfH_GC
B_IFN
B_preGC
B_mem
B_naive
DC_cDC1
Macrophages_M1
B_activated
B_plasma
DC_cDC2
VSMC
Endo
Monocytes
NK
T_CD8+_cytotoxic
T_CD4+_TfH
T_CD8+_CD161+
Macrophages_M2
Mast
NKT
DC_pDC
ILC
T_TIM3+
DC_CCR7+
T_TfR
T_Treg
T_CD4+
T_CD4+_naive
T_CD8+_naive
fact_1
fact_1
fact_9
fact_8
fact_7
fact_6
fact_5
fact_4
fact_3
fact_2
fact_1
fact_0
0.8
0.6
0.4
0.2
0.0

Estimate cell-type specific expression of every gene in the spatial data (needed for NCEM)

The cell-type specific expression of every gene at every spatial location in the spatial data enables learning cell communication with NCEM model using Visium data (https://github.com/theislab/ncem).

To derive this, we adapt the approach of estimating conditional expected expression proposed by RCTD (Cable et al) method.

With cell2location, we can look at the posterior distribution rather than just point estimates of cell type specific expression (see `mod_c2l.samples.keys()` and next section on using full distribution).

Note that this analysis requires substantial amount of RAM memory and thefore doesn't work on free Google Colab (12 GB limit).

In [81]:
```
mod_c2l
```

```
cell2location model with the following params:
n_factors: 34
n_batch: 1
Training status: Trained
```

Out[81]:

In [82]:
```
print(list(mod_c2l.samples.keys()))
```

```
['post_sample_q05', 'post_sample_q50', 'post_sample_q95', 'post_sample_q00
01']
```

In [83]:
```
'''
# Compute expected expression per cell type
expected_dict = mod_c2l.module.model.compute_expected_per_cell_type(
    mod_c2l.samples["post_sample_q05"], mod_c2l.adata_manager
)

# Add to anndata layers
for i, n in enumerate(mod_c2l.factor_names_):
    adata_vis.layers[n]= expected_dict['mu'][i]
'''

# Save anndata object with results (adata_file_sp, not adata_ref_filename
adata_vis.write(adata_file_sp)
adata_file_sp
```

Out[83]:
```
PosixPath('/home/flavio/uv/squidpy/results/lymph_nodes_analysis/cell2loc
ation_map/sp.h5ad')
```

In [84]:
```
adata_vis
```

Out[84]:
```
AnnData object with n_obs × n_vars = 4035 × 10217
    obs: 'in_tissue', 'array_row', 'array_col', 'sample', '_indices', '_
scvi_batch', '_scvi_labels', 'B_Cycling', 'B_GC_DZ', 'B_GC_LZ', 'B_GC_pr
ePB', 'B_IFN', 'B_activated', 'B_mem', 'B_naive', 'B_plasma', 'B_preGC',
'DC_CCR7+', 'DC_cDC1', 'DC_cDC2', 'DC_pDC', 'Endo', 'FDC', 'ILC', 'Macro
phages_M1', 'Macrophages_M2', 'Mast', 'Monocytes', 'NK', 'NKT', 'T_CD4
+', 'T_CD4+_TfH', 'T_CD4+_TfH_GC', 'T_CD4+_naive', 'T_CD8+_CD161+', 'T_C
D8+_cytotoxic', 'T_CD8+_naive', 'T_TIM3+', 'T_TfR', 'T_Treg', 'VSMC', 'l
eiden', 'region_cluster', 'mean_nUMI_factorsfact_0', 'mean_nUMI_factorsf
act_1', 'mean_nUMI_factorsfact_2', 'mean_nUMI_factorsfact_3', 'mean_nUMI
_factorsfact_4', 'mean_nUMI_factorsfact_5', 'mean_nUMI_factorsfact_6',
'mean_nUMI_factorsfact_7', 'mean_nUMI_factorsfact_8', 'mean_nUMI_factors
fact_9', 'mean_nUMI_factorsfact_10', 'mean_nUMI_factorsfact_11', 'region
_name'
    var: 'feature_types', 'genome', 'symbol', 'MT_gene'
    uns: '_scvi_manager_uuid', '_scvi_uuid', 'leiden', 'mod', 'mod_coloc
_n_fact11', 'mod_coloc_n_fact12', 'neighbors', 'region_cluster_colors',
'sample_colors', 'spatial', 'umap'
    obsm: 'MT', 'X_umap', 'means_cell_abundance_w_sf', 'q0001_cell_abund
ance_w_sf', 'q05_cell_abundance_w_sf', 'q50_cell_abundance_w_sf', 'q95_c
ell_abundance_w_sf', 'spatial', 'stds_cell_abundance_w_sf'
    obsp: 'connectivities', 'distances'
    layers: 'B_Cycling', 'B_GC_DZ', 'B_GC_LZ', 'B_GC_prePB', 'B_IFN', 'B
_activated', 'B_mem', 'B_naive', 'B_plasma', 'B_preGC', 'DC_CCR7+', 'DC_
cDC1', 'DC_cDC2', 'DC_pDC', 'Endo', 'FDC', 'ILC', 'Macrophages_M1', 'Mac
rophages_M2', 'Mast', 'Monocytes', 'NK', 'NKT', 'T_CD4+', 'T_CD4+_TfH',
'T_CD4+_TfH_GC', 'T_CD4+_naive', 'T_CD8+_CD161+', 'T_CD8+_cytotoxic', 'T
_CD8+_naive', 'T_TIM3+', 'T_TfR', 'T_Treg', 'VSMC', None (.X)
```

Plotting cell-type specific expression of genes in spatial coordinates.
Below we plot the cell-type specific expression of genes (rows, second to last columns) compared to total expression of those genes (first column).

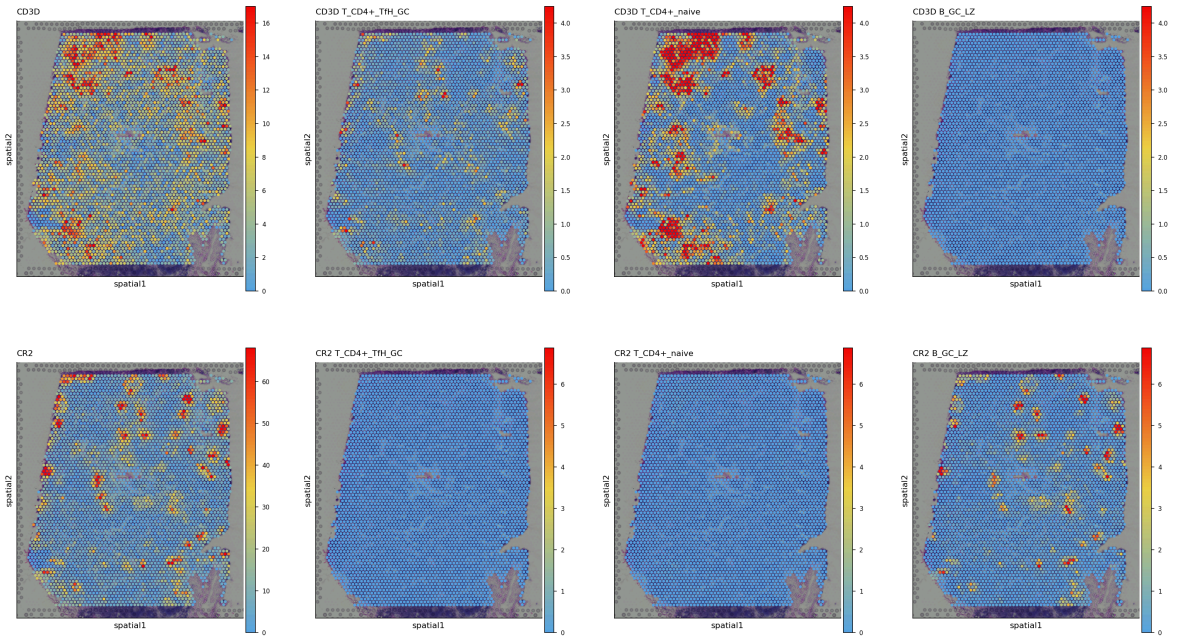
Here we highlight *CD3D*, pan T-cell marker expressed by 2 subtypes of T cells in distinct locations but not expressed by co-located B cells, that instead express *CR2* gene.

In [85]:
```
import libs.cell2location_lib as c2l
importlib.reload(c2l)
```

Out[85]:
```
<module 'libs.cell2location_lib' from '/home/flavio/uv/squidpy/src/libs/
cell2location_lib.py'>
```

In [86]:
```
# list cell types and genes for plotting
ctypes = ['T_CD4+_TfH_GC', 'T_CD4+_naive', 'B_GC_LZ']
genes = ['CD3D', 'CR2']

with mpl.rc_context({'axes.facecolor':  'black'}):
    # select one slide
    slide = select_slide(adata_vis, 'V1_Human_Lymph_Node')

    # from tutorial_utils import plot_genes_per_cell_type
    c2l.plot_genes_per_cell_type(slide, genes, ctypes)
```

```
/home/flavio/uv/squidpy/src/libs/cell2location_lib.py:287: FutureWarning:
The function spatial is deprecated and will be removed in the future. Use
:func:`squidpy.pl.spatial_scatter` instead.
  sc.pl.spatial(
/home/flavio/uv/squidpy/src/libs/cell2location_lib.py:305: FutureWarning:
The function spatial is deprecated and will be removed in the future. Use
:func:`squidpy.pl.spatial_scatter` instead.
  sc.pl.spatial(
```



In [87]:
```
slide = select_slide(adata_vis, 'V1_Human_Lymph_Node')
```

In [88]:
```
slide.var
```

Out[88]:

gene_ids	feature_types	genome	symbol	MT_gene
ENSG00000000419	Gene Expression	GRCh38	DPM1	False
ENSG00000000457	Gene Expression	GRCh38	SCYL3	False
ENSG00000000938	Gene Expression	GRCh38	FGR	False
ENSG00000000971	Gene Expression	GRCh38	CFH	False
ENSG00000001036	Gene Expression	GRCh38	FUCA2	False
...	...	...	...	...
ENSG00000280789	Gene Expression	GRCh38	PAGR1	False
ENSG00000280987	Gene Expression	GRCh38	MATR3	False
ENSG00000281103	Gene Expression	GRCh38	TRG-AS1	False
ENSG00000282608	Gene Expression	GRCh38	ADORA3	False
ENSG00000282851	Gene Expression	GRCh38	BISPR	False

10217 rows × 4 columns

In [89]:
```
slide.layers
```

Out[89]:
```
Layers with keys: 'B_Cycling', 'B_GC_DZ', 'B_GC_LZ', 'B_GC_prePB', 'B_IF
N', 'B_activated', 'B_mem', 'B_naive', 'B_plasma', 'B_preGC', 'DC_CCR7
+', 'DC_cDC1', 'DC_cDC2', 'DC_pDC', 'Endo', 'FDC', 'ILC', 'Macrophages_M
1', 'Macrophages_M2', 'Mast', 'Monocytes', 'NK', 'NKT', 'T_CD4+', 'T_CD4
+_TfH', 'T_CD4+_TfH_GC', 'T_CD4+_naive', 'T_CD8+_CD161+', 'T_CD8+_cytoto
xic', 'T_CD8+_naive', 'T_TIM3+', 'T_TfR', 'T_Treg', 'VSMC', None
```

In [ ]:

Note that `plot_genes_per_cell_type` function often need customization so it is not included into cell2location package - you need to copy it from https://github.com/BayraktarLab/cell2location/blob/master/docs/notebooks/tutorial_utils.p to use on your system.

Advanced use

Working with the posterior distribution and computing arbitrary quantiles

In addition to the posterior distribution mean, std and quantiles presented earlier in the notebook you can fetch an arbitrary number of samples from the posterior distribution. To limit memory use, it could be beneficial to select particular varibles in the model.

Note that this analysis requires substantial amount RAM memory and thefore doesn't work on Google Colab.

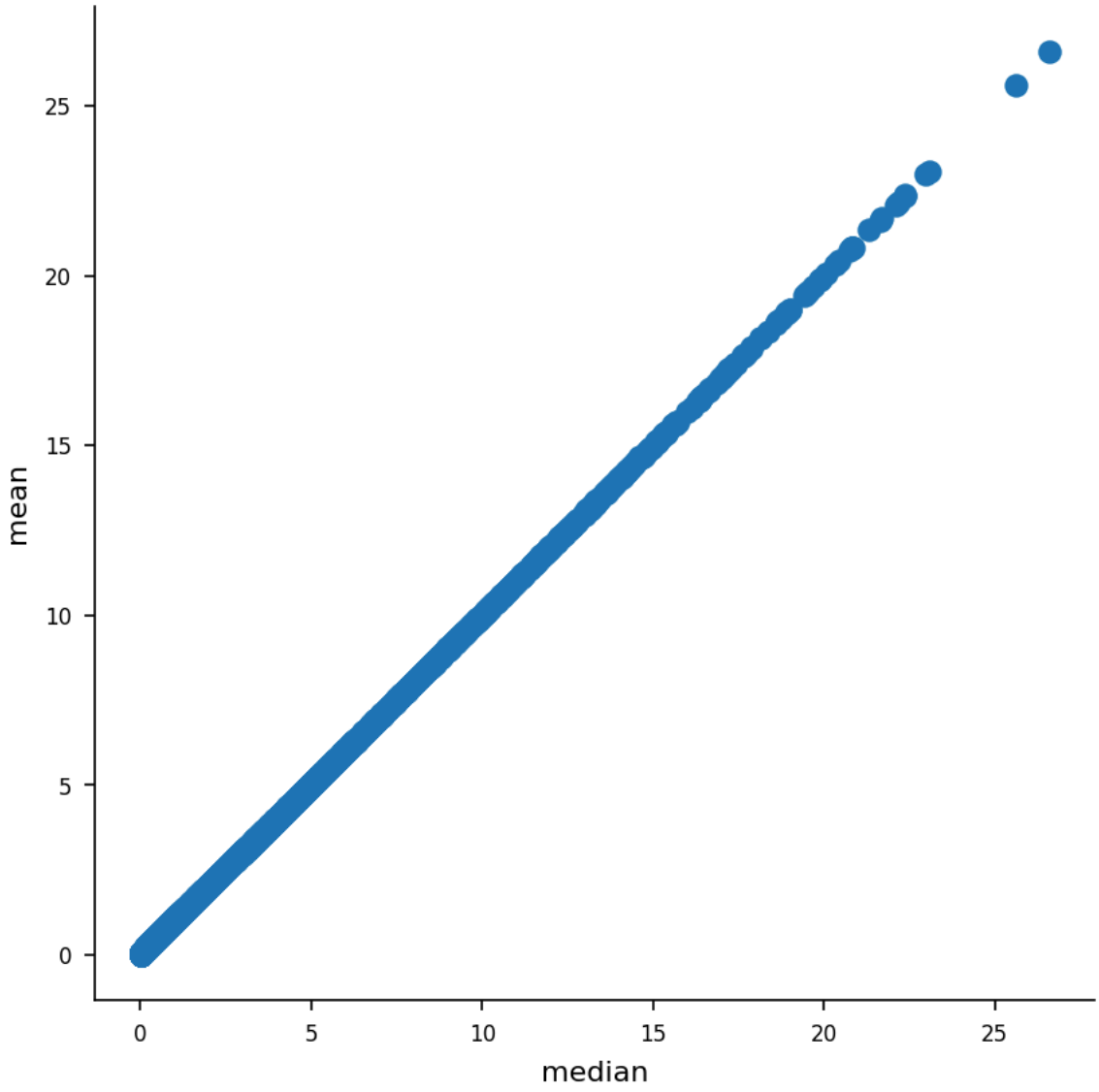
In [90]:
```
# Get posterior distribution samples for specific variables
samples_w_sf = mod_c2l.sample_posterior(num_samples=1000, accelerator='gp
                                        batch_size=2020,
                                        return_sites=['w_sf', 'm_g', 'u_sf_mR
# samples_w_sf['posterior_samples']contains 1000 samples as arrays with d
samples_w_sf['posterior_samples']['w_sf'].shape
```

```
Sampling local variables, batch:   0%|          | 0/2 [00:00<?, ?it/s]
Sampling global variables, sample:   0%|          | 0/999 [00:00<?, ?it/s]
```

Out[90]:
```
(1000, 4035, 34)
```

Finally, it could be useful to compute arbitrary quantiles of the posterior distribution.

In [91]:
```
# Compute any quantile of the posterior distribution
medians = mod_c2l.posterior_quantile(q=0.5, batch_size=mod_c2l.adata.n_ob

with mpl.rc_context({'axes.facecolor':  'white',
                     'figure.figsize': [5, 5]}):
    if 'post_sample_means' in mod_c2l.samples:
        means_w_sf = mod_c2l.samples['post_sample_means']['w_sf']
    else:   # quantile-only export: use the posterior samples drawn in th
        means_w_sf = samples_w_sf['posterior_samples']['w_sf'].mean(0)

    plt.scatter(medians['w_sf'].flatten(), np.asarray(means_w_sf).flatten
    plt.xlabel('median')
    plt.ylabel('mean');
```



Modules and their versions used for this analysis

Useful for debugging and reporting issues.

In [92]:
```
cell2location.utils.list_imported_modules()
```

```
sys 3.12.12 (main, Feb 12 2026, 00:42:14) [Clang 21.1.4 ]
re 2.2.1
ipaddress 1.0
ipykernel._version 6.31.0
json 2.0.9
jupyter_client._version 8.10.0
platform 1.0.8
zlib 1.0
zmq.sugar.version 27.2.0
zmq.sugar 27.2.0
zmq 27.2.0
logging 0.5.1.2
traitlets._version 5.16.1
traitlets 5.16.1
jupyter_core.version 5.9.1
jupyter_core 5.9.1
tornado 6.5.8
_ctypes 1.1.0
ctypes 1.1.0
colorama 0.4.6
_curses b'2.2'
socketserver 0.4
argparse 1.1
dateutil._version 2.9.0.post0
dateutil 2.9.0.post0
six 1.17.0
_decimal 1.70
decimal 1.70
msgpack 1.2.2
platformdirs.version 4.11.8
platformdirs 4.11.8
_csv 1.0
csv 1.0
jupyter_client 8.10.0
ipykernel 6.31.0
IPython.core.release 9.17.1
pygments 2.21.0
IPython 9.17.1
IPython.core.crashhandler 9.17.1
comm 0.2.3
psutil 7.2.2
packaging 26.3
IPython.core.magics.code 9.17.1
debugpy.public_api 1.8.21
debugpy 1.8.21
xmlrpc.client 3.12
http.server 0.6
_pydevd_frame_eval.vendored.bytecode 0.13.0.dev
_pydev_bundle.fsnotify 0.1.5
pydevd 3.4.1
sqlite3.dbapi2 2.6.0
sqlite3 2.6.0
ipywidgets._version 8.1.9
ipywidgets 8.1.9
pydantic_core._pydantic_core 2.41.5
pydantic_core 2.41.5
pydantic.version 2.12.5
pydantic 2.12.5
annotated_types 0.8.0
pydantic_settings.version 2.15.0
```

```
pydantic_settings 2.15.0
numpy.version 2.5.3
numpy._core._multiarray_umath 3.1
numpy._core 2.5.3
numpy.linalg._umath_linalg 0.1.5
numpy 2.5.3
h5py.version 3.16.0
h5py 3.16.0
pytz 2026.5
pyarrow._generated_version 25.0.1
cloudpickle 3.1.2
pyarrow 25.0.1
tarfile 0.9.0
pandas._version_meson 2.3.3
pandas 2.3.3
zarr._version 3.3.0
yaml 6.0.3
donfig 0.8.1.post1
numcodecs.version 0.16.5
numcodecs.blosc 1.21.7.dev
numcodecs.zstd 1.5.6
numcodecs.lz4 1.10.0
numcodecs 0.16.5
upath._version 0.3.10
upath 0.3.10
fsspec._version 2026.7.0
fsspec 2026.7.0
zarr 3.3.0
natsort 8.4.0
scipy.version 1.18.1
scipy 1.18.1
scipy._external.array_api_compat 1.14.0
numpy.f2py.__version__ 2.5.3
numpy.f2py.cfuncs <module 'numpy.f2py.__version__' from '/home/flavio/uv/s
quidpy/.venv/lib/python3.12/site-packages/numpy/f2py/__version__.py'>
numpy.f2py.auxfuncs <module 'numpy.f2py.__version__' from '/home/flavio/u
v/squidpy/.venv/lib/python3.12/site-packages/numpy/f2py/__version__.py'>
numpy.f2py.cb_rules <module 'numpy.f2py.__version__' from '/home/flavio/u
v/squidpy/.venv/lib/python3.12/site-packages/numpy/f2py/__version__.py'>
charset_normalizer.version 3.5.1
charset_normalizer 3.5.1
numpy.f2py.crackfortran <module 'numpy.f2py.__version__' from '/home/flavi
o/uv/squidpy/.venv/lib/python3.12/site-packages/numpy/f2py/__version__.p
y'>
numpy.f2py.capi_maps <module 'numpy.f2py.__version__' from '/home/flavio/u
v/squidpy/.venv/lib/python3.12/site-packages/numpy/f2py/__version__.py'>
numpy.f2py.f90mod_rules  1.27
numpy.f2py.common_rules <module 'numpy.f2py.__version__' from '/home/flavi
o/uv/squidpy/.venv/lib/python3.12/site-packages/numpy/f2py/__version__.p
y'>
numpy.f2py.use_rules  1.3
numpy.f2py.rules <module 'numpy.f2py.__version__' from '/home/flavio/uv/sq
uidpy/.venv/lib/python3.12/site-packages/numpy/f2py/__version__.py'>
numpy.f2py.f2py2e <module 'numpy.f2py.__version__' from '/home/flavio/uv/s
quidpy/.venv/lib/python3.12/site-packages/numpy/f2py/__version__.py'>
numpy.f2py <module 'numpy.f2py.__version__' from '/home/flavio/uv/squidp
y/.venv/lib/python3.12/site-packages/numpy/f2py/__version__.py'>
numpy.core 2.5.3
scipy._external.array_api_compat.numpy 2.5.3
scipy._external.array_api_extra 0.10.1
```

```
toolz 1.1.0
tlz 1.1.0
dask._version 2026.8.0
tblib 3.2.2
markupsafe 3.0.3
jinja2 3.1.6
dask 2026.8.0
xarray.tutorial master
xarray 2026.7.0
sparse._version 0.19.2
llvmlite 0.49.0
numba.cloudpickle 3.1.1
numba.misc.appdirs 1.4.1
numba 0.67.0
sparse 0.19.2
scipy._lib._uarray 0.8.8.dev0+aa94c5a4.scipy
scipy.linalg._fblas 2.5.2
scipy.linalg._flapack 2.5.2
anndata 0.13.3.post0
matplotlib._version 3.10.9
PIL._version 11.3.0
PIL 11.3.0
PIL._deprecate 11.3.0
defusedxml 0.7.1
xml.etree.ElementTree 1.3.0
PIL.Image 11.3.0
pyparsing 3.3.2
cycler 0.12.1
kiwisolver._cext 1.5.1
kiwisolver 1.5.1
matplotlib 3.10.9
urllib.request 3.12
joblib.externals.loky 3.6.0
joblib 1.6.0
narwhals 2.26.0
sklearn.externals.array_api_compat 1.13.0
sklearn.externals.array_api_extra 0.8.2
sklearn.externals.array_api_compat.numpy 2.5.3
threadpoolctl 3.6.0
sklearn.utils._repr_html.base 1.9.0
sklearn.base 1.9.0
sklearn.utils._show_versions 1.9.0
sklearn 1.9.0
scanpy 1.12.4
torch._vendor.packaging 23.2
torch.version 2.14.1+cu130
torch.torch_version 2.14.1+cu130
cuda.bindings._version 13.4.3
cuda.bindings 13.4.3
cuda.pathfinder._version 1.8.3
cuda.pathfinder 1.8.3
torch.backends.cudnn 92400
torch.backends.cusparselt 801
opt_einsum._version 3.4.0
opt_einsum 3.4.0
torch.cuda.nccl (2, 30, 7)
tqdm.version 4.70.0
tqdm.cli 4.70.0
tqdm 4.70.0
torch.ops <module 'torch.ops.__version__' from 'torch.ops'>
```

```
torch.classes torch.classes torch 2.14.1+cu130
lightning.__version__ 2.6.6
lightning_utilities.__about__ 0.15.3
lightning_utilities 0.15.3
torchmetrics.__about__ 1.9.0
torchmetrics 1.9.0
mpmath 1.3.0
sympy.release 1.14.0
sympy.multipledispatch 0.4.9
sympy 1.14.0
triton.runtime.cache 3.8.0
triton.compiler.compiler 3.8.0
triton 3.8.0
lightning.fabric 2.6.6
lightning.pytorch 2.6.6
lightning 2.6.6
absl 2.5.1
ml_collections 1.1.0
docrep 0.3.2
mudata._version 0.4.1
mudata 0.4.1
attr 26.1.0
pyro._version 1.9.1
pyro 1.9.1
statsmodels._version 0.15.0
statsmodels 0.15.0
scvi 1.5.1
cffi 2.1.1
_cffi_backend 2.1.1
pycparser 3.00
pynndescent 0.6.0
umap 0.5.12
cell2location 0.1.5
shapely 2.1.2
certifi 2026.07.22
pyproj 3.8.0
geopandas 1.1.4
multiscale_spatial_image.__about__ 2.0.3
xarray_dataclass.__about__ 3.0.0
xarray_dataclass 3.0.0
spatial_image 1.2.3
multiscale_spatial_image 2.0.3
ome_zarr_models 1.8.1
wrapt 2.4.0
deprecated 1.3.1
lazy_loader 0.5
skimage 0.26.0
array_api_compat 1.15.0
networkx 3.6.1
transformnd 0.7.2
ome_zarr._version 0.19.2
ome_zarr 0.19.2
spatialdata 0.8.0
pooch._version 1.9.0
pooch.downloaders 1.9.0
pooch v1.9.0
more_itertools 11.1.0
annsel 0.1.3
numba_progress._version 1.2.0
numba_progress 1.2.0
```

```
numba.types.np 2.5.3
numba.types.ctypes 1.1.0
numba.types.logging 0.5.1.2
rustworkx.rustworkx 0.18.1
rustworkx 0.18.1
validators 0.35.0
imagecodecs.imagecodecs 2025.11.11
imagecodecs 2025.11.11
tifffile.tifffile 2026.9.9
tifffile 2026.9.9
imageio 2.37.4
seaborn.external.husl 2.1.0
seaborn.external.appdirs 1.4.4
seaborn 0.13.2
matplotlib_scalebar._version 0.9.0
matplotlib_scalebar 0.9.0
squidpy 0.1.dev34+gb2ed182e8.d20261020
urllib3._version 2.7.0
urllib3.util.ssl_match_hostname 3.5.0.1
urllib3.connection 2.7.0
urllib3 2.7.0
requests.packages.urllib3._version 2.7.0
requests.packages.urllib3.util.ssl_match_hostname 3.5.0.1
requests.packages.urllib3.connection 2.7.0
requests.packages.urllib3 2.7.0
idna.idnadata 17.0.0
idna.package_data 3.19
idna 3.19
requests.packages.idna.idnadata 17.0.0
requests.packages.idna.package_data 3.19
requests.packages.idna 3.19
requests.packages.charset_normalizer.version 3.5.1
requests.packages.chardet.version 3.5.1
requests.packages.charset_normalizer 3.5.1
requests.packages.chardet 3.5.1
requests.__version__ 2.34.2
requests.utils 2.34.2
requests 2.34.2
zstandard.backend_c 0.25.0
zstandard 0.25.0
matplotlib_inline 0.2.2
plotly 7.1.0
wcwidth 0.8.3
texttable 1.7.0
igraph.version 1.0.0
igraph 1.0.0
leidenalg.version 0.11.0
leidenalg 0.11.0
fastjsonschema.version 2.22.2
fastjsonschema 2.22.2
attrs 26.1.0
rfc3986_validator 0.1.1
lark 1.3.1
rfc3339_validator 0.1.4
jsonpointer 3.1.1
arrow._version 1.4.0
arrow 1.4.0
jsonschema 4.26.0
nbformat._version 5.11.1
```

```
nbformat 5.11.1
libs.cell2location_lib 1.0.0
```

```
/home/flavio/uv/squidpy/.venv/lib/python3.12/site-packages/cell2location/u
tils/__init__.py:29: FutureWarning: `__version__` is deprecated, use `impo
rtlib.metadata.version('anndata')` instead.
  print(module, sys.modules[module].__version__)
/home/flavio/uv/squidpy/.venv/lib/python3.12/site-packages/cell2location/u
tils/__init__.py:29: FutureWarning: `__version__` is deprecated, use `impo
rtlib.metadata.version('scanpy')` instead
  print(module, sys.modules[module].__version__)
```

In [93]:
```
cell2location.__version__
```

Out[93]:
```
'0.1.5'
```